

Got it — I'll teach this like your senior Django mentor. I'll assume you know Python and web basics, and I'll treat every Django-specific file/command as new. For each item I'll follow your requested micro-structure (literal code, why, internals, when loaded, failure modes, common beginner mistake). I'll cover PART 1 → PART 3 (Mental model, Project files, Commands). After each PART you get 2–3 short active-check questions plus answers.

Part 4+ (templates, static, media, ORM deep dives, advanced topics) I can add next if you want — tell me when.

SECTION: PART 1 — MENTAL MODEL

1. What literally happens when a user types a URL and hits Enter (end-to-end)
2. What happens when you run `manage.py runserver`
3. MVT architecture with a concrete request
4. How settings/apps/project relate

PART 1 — MENTAL MODEL

A. What literally happens when a user types a URL and hits Enter (end-to-end)

What is it, literally?

A chain of handoffs from the browser to DNS → TCP/TLS → HTTP request → server process → Django internals → response returned.

Minimal ASCII flow:

```
```text
```

```
Browser (http request) -> DNS resolver (domain -> IP) -> TCP handshake (client <-> server) -> TLS handshake
(if https) -> HTTP request bytes sent -> Web server (Nginx/Dev server) -> WSGI/ASGI server
(Gunicorn/Uvicorn) -> Django process -> Middleware (request phase) -> URL resolver (urls.py)
```

```
-> View (views.py)
```

```
-> ORM/Services (models.py, queries) -> Template render or JSON serialization -> HttpResponse object ->
Middleware (response phase) -> WSGI/ASGI -> Web server -> Browser (response)
```

```
```
```

Why does Django need this explained concretely?

So you can map where to debug latency, 404s, auth issues, template errors or DB problems. Interviews often ask "what happens when you type `example.com`" — this map is the canonical answer.

How does it work internally? (hand-offs, what runs)

- Browser resolves domain: local resolver → recursive resolver → authoritative DNS; returns

IP.

- Browser opens TCP 3-way handshake (SYN, SYN-ACK, ACK) to the IP:port (80/443).
- If HTTPS, TLS handshake negotiates symmetric keys (ClientHello -> ServerHello -> certificates -> keys).
- HTTP request bytes are sent (headers + body) to server process.
- On production: Nginx (reverse proxy) accepts connection, optionally terminates TLS, proxies to upstream worker over HTTP or Unix socket. On dev: Django development server (runserver) receives it.
- WSGI (sync) or ASGI (async) server receives request and calls Django application callable.
- Django bootstrap for request:
- settings and app registry were loaded at startup (see runserver sequence below).
- request enters middleware chain: each `middleware.request(request)` hook may run; it can short-circuit and return a response.
- URL resolver inspects `urlpatterns` (root -> includes) to find first matching pattern; captures path params.
- Resolver loads target view and calls it (function or view callable from Class Based View `as_view()` wrapper). At this point `request.user` was attached by AuthenticationMiddleware earlier.
- View executes: validating inputs (forms/serializers), calling service layer, performing ORM queries.
- ORM translates QuerySet operations to SQL, sends to DB via DB driver (psycopg2 for Postgres), receives rows, produces model instances.
- If view returns template response, Django renders template by loading template file, compiling nodes, evaluating with context, escaping content, producing HTML string.
- View returns `HttpResponse` (status, headers, body). Response-phase middleware runs in reverse order; it can modify headers, cookies, streaming, or do logging.
- WSGI/ASGI server receives response and writes bytes out. Reverse-proxy returns to client. Browser parses and renders.

When does each handoff happen? (startup vs per-request)

- DNS/TCP/TLS: per-request from client side (outside Django).
- WSGI/ASGI server and Django import/app registry initialization: at process start (startup).
- Middleware URL resolution, views, ORM, template rendering: per-request.
- Connection pooling (DB): pools are created at startup and used per-request.

What breaks if missing or wrong?

- DNS misconfiguration -> client can't resolve domain (browser shows DNS error).
- TLS/cert misconfigured -> browser shows insecure site or fails handshake.
- Nginx misproxy settings -> 502 Bad Gateway.
- Missing URL pattern -> 404.
- Error in view template -> `TemplateDoesNotExist` or `TemplateSyntaxError`.
- ORM mismatch (migrations not applied) -> `FieldDoesNotExist` / column missing or DB errors.
- Middleware order wrong -> authentication/session not available (e.g., `AuthenticationMiddleware` must come after `SessionMiddleware`).

A common beginner mistake

Assuming Django is responsible for DNS/TLS — those are infra. Another frequent error: putting database-heavy work in template (blocking request and causing timeouts).

B. What happens when you run `python manage.py runserver`

What is it, literally?

A development server process that boots Django for local development with auto-reload. Example minimal invocation:

```
```bash
```

```
python manage.py runserver 0.0.0.0:8000
```

```
```
```

Why does Django need this command?

It provides a quick local HTTP server that loads your Django app and helps you iterate (auto-reload on code change).

How does it work internally?

- `manage.py` sets `DJANGO_SETTINGS_MODULE` and calls Django management command runner.
- `runserver` command:
- Bootstraps Django: sets up settings, logging, app registry (loads `INSTALLED_APPS` and `AppConfig.ready()`).
- Loads `URLconf` (`ROOT_URLCONF`). Validates.
- Initializes autoreload watcher: runs a reloader that monitors files and restarts workers on change.
- Starts a single-process threaded HTTP server in dev (not suitable for production): for Python 3 it uses `django.utils.autoreload` + `socketserver` or uses ASGI server if configured to run async dev server.
- Optionally prints server info and handles signals (Ctrl-C).
- On incoming requests the same request cycle runs (middleware -> url resolver -> view -> response).

When does Django load/use components here?

- Settings and `INSTALLED_APPS` are loaded on the `runserver` startup.
- Migration state, admin models, signal handlers from apps are set up during app registry initialization at startup.
- Template loaders, staticfiles finders are set up at startup but used per-request or by `collectstatic`.

What breaks if missing/wrong?

- If `DJANGO_SETTINGS_MODULE` wrong -> `ImportError` at startup.
- If settings reference missing env vars -> crash on startup.
- If `INSTALLED_APPS` contains an app that raises on import -> startup failure.

- If `ALLOWED_HOSTS` and `DEBUG=False` -> runserver will reject host headers (in dev, `DEBUG=True` by default).

A common beginner mistake

Relying on runserver behavior for production performance/security (e.g., serving static files directly or ignoring using Gunicorn/Nginx).

C. MVT (Model-View-Template) — concrete request example walked through each layer

What is it, literally?

- Model: Python class inheriting `models.Model` mapping to DB.
- View: function/class handling request.
- Template: DTL file rendering HTML.

Minimal concrete snippet: `models.py`

```
``python
```

```
from django.db import models
class Transaction(models.Model):
    user = models.ForeignKey("auth.User", on_delete=models.CASCADE)
    amount = models.DecimalField(max_digits=10, decimal_places=2)
    date = models.DateField()
    note = models.TextField(blank=True)
```

```
```
```

`views.py`

```
``python
```

```
from django.shortcuts import render
def tx_list(request):
 qs = Transaction.objects.filter(user=request.user).order_by('-date')[:50]
 context = {"transactions": qs}
 return render(request, "finance/tx_list.html", context)
```

```
```
```

`templates/finance/tx_list.html`

```
``django
```

```
{% extends "base.html" %}
{% block content %}
```

```
{% for tx in transactions %}
```

- ```
{{ tx.date }} - {{ tx.amount }} - {{ tx.note }}
```
- ```
{% endfor %} ## {% endblock %} ## ``
```

Why does Django need MVT specifically?

It provides separation: Models hold data, Views orchestrate logic/formatting, Templates present UI. This keeps concerns distinct and code testable.

How does it work internally? (execution order)

- Request -> URL matches `tx_list`.
- View `tx_list` is called with `request`.
- The view executes `Transaction.objects.filter(...)` — QuerySet lazy, when iterated/templates evaluated it triggers database SELECT with WHERE `user_id = ... ORDER BY date DESC LIMIT 50`.
- QuerySet returns model instances; view builds context dict and calls template renderer.
- Template engine compiles and renders HTML, escapes variables, returns `HttpResponse`.

When does Django load/use each piece?

- Models are imported at startup (app registry); queries happen per-request.
- Template compilation may be cached; actual rendering happens per-request unless cached.
- Views are callables invoked per-request.

What breaks if missing/wrong?

- If model field name changed but view still uses old name -> `AttributeError` at template rendering or view.
- If template path wrong -> `TemplateDoesNotExist` when view calls render.
- If view forgets to restrict `user=request.user` -> data leak.

A common beginner mistake

Returning QuerySet to template without slicing or prefetching leads to N+1 issues in loops when referencing related objects.

D. How project-level settings, apps, and the project relate (small diagram)

What is it, literally?

`settings.py` configures `INSTALLED_APPS`; each app is a Python package with models/views/templates; project wires apps together via `ROOT_URLCONF`.

ASCII diagram:

```
```text
```

settings.py

```
INSTALLED_APPS = [
```

```
"django.contrib.auth",
```

```
"finance.apps.FinanceConfig", <-- app registered "reports.apps.ReportsConfig",
```

```
]
```

project root (urls.py)

-> includes finance.urls, reports.urls

## App (finance)

- models.py
- views.py
- urls.py
- templates/finance/\*.html

```
```
```

Why does Django need this relationship?

Django's app registry uses settings to register models, admin, signals, migrations and template/static lookup.

How does it work internally?

At startup Django imports settings, then calls `django.setup()` which constructs **Apps** registry: it imports each **AppConfig** (from apps.py) and runs ready hooks. That populates the global model registry and admin.

When loaded?

At process startup (runserver/gunicorn worker start or management command invocation). Some lazy imports happen later.

What breaks if missing/wrong?

- Missing app from `INSTALLED_APPS` -> migrations won't be considered, admin won't show models, templates/static under app not found by app-specific finders.
- Import error in `AppConfig.ready()` -> startup failure.

Common beginner mistake

Editing `INSTALLED_APPS` after startup in code or using wrong dotted path for `AppConfig`.

```
---
```

PART 1 — CHECK YOURSELF (2–3 short questions + answers)

Q1 — Explain step-by-step what happens (inside Django) between URL resolution and template output for `tx_list` above. A1 — URL resolver matches path -> view called -> QuerySet built -> iteration triggers DB SELECT -> ORM returns model instances -> view creates context -> `render()` finds template -> template engine compiles + renders with context -> returns `HttpResponse`.

Q2 — If your template throws `TemplateDoesNotExist` only on production, what are 3 likely causes? A2 — (1) `collectstatic`/template files not deployed or `TEMPLATE_DIRS` misconfigured; (2) app missing in

INSTALLED_APPS, so app template loader not found; (3) wrong path/filename differing by case on case-sensitive FS in prod.

Q3 — Why must `AuthenticationMiddleware` come after `SessionMiddleware` in

MIDDLEWARE?

A3 — `AuthenticationMiddleware` relies on session data (`request.session`) to load `request.user`. If `SessionMiddleware` hasn't run, `request.session` will be missing and auth won't work.

If this structure and level are good, I'll continue with PART 2 — PROJECT STRUCTURE (`manage.py`, `settings.py`, project-level and app-level `urls.py`, `views.py`, `models.py`, `admin.py`, `apps.py`, `middleware.py`, `forms.py`, `wsgi.py`, `asgi.py`) and then PART 3 (commands) exactly in the micro-structure you demanded. Confirm and I'll produce the full Part 2+3 now.

Understood — I'll generate the remainder of Prompt 1 now (PART 2: Project Structure; PART 3: Commands), using the exact same style and depth as Part 1. No quizzing. Each file/command gets the micro-structure you requested:

- What is it, literally? (real minimal code / syntax)
- Why does Django need this specifically?
- How does it work internally? (what Django does with it under the hood)
- When does Django load/use it? (startup vs per-request)
- What breaks if it's missing or wrong?
- A common beginner mistake
- Short flow / diagram where useful

I'll keep examples anchored to your finance-tracker context where helpful.

PART 2 — PROJECT STRUCTURE (all core Django files)

`manage.py`

What is it, literally?

- A thin command-line entry-point script created at project bootstrap.

Example (auto-generated):

```
```python
```

```
#!/usr/bin/env python import os import sys
```

```
if name == "main": os.environ.setdefault("DJANGO_SETTINGS_MODULE", "finance_tracker.settings") from
django.core.management import execute_from_command_line execute_from_command_line(sys.argv)

'''
```

Why does Django need this file?

- It sets DJANGO\_SETTINGS\_MODULE for local commands and delegates to Django's management command dispatcher so you can run runserver, migrate, shell, etc., with project context.

How does it work internally?

- It sets the environment variable, then imports Django's execute\_from\_command\_line which bootstraps Django, loads settings, and runs the requested management command.

When is it used?

- Only when you invoke `python manage.py <command>` — every development/management operation.

What breaks if it's missing/wrong?

- You can still call `django-admin` but `manage.py` is the convenient per-project entry. If it points at wrong settings module or sets wrong env, commands will load incorrect config or fail to import settings.

Common beginner mistake

- Hard-coding settings or paths in `manage.py` or committing a `manage.py` that points to local-only settings.

settings.py

-----

What is it, literally?

- A Python module with assignments that configure Django: INSTALLED\_APPS,

MIDDLEWARE, DATABASES, TEMPLATES, STATIC\_, MEDIA\_,  
AUTH\_USER\_MODEL,

SECRET\_KEY, DEBUG, ALLOWED\_HOSTS, etc. Minimal snippet:

```
```python
```

```
BASE_DIR = Path(file).resolve().parent.parent SECRET_KEY = env('SECRET_KEY') DEBUG = env.bool('DEBUG',
default=True)
```

```
INSTALLED_APPS = [
```

```
'django.contrib.admin',
```

```
'django.contrib.auth',
```



```
'finance',
'reports',
]

DATABASES = {
'default': env.db('DATABASE_URL', default='sqlite:///db.sqlite3')
}

STATIC_URL = '/static/' MEDIA_URL = '/media/'
'''
```

Why does Django need it?

- Central configuration that controls how the framework and your apps behave.

How does it work internally?

- At startup, Django reads DJANGO_SETTINGS_MODULE and imports this module. Django core modules and apps read relevant settings during setup (app registry, middleware, DB connections, templates, staticfiles finders).

When is it loaded/used?

- Loaded at process startup (runserver, gunicorn workers, management commands). Settings may be read lazily by some subsystems but initial import is at boot.

What breaks if missing/wrong?

- Missing DJANGO_SETTINGS_MODULE or import errors => startup failure. Wrong DB settings => DB connection errors. DEBUG left True in prod => sensitive info leakage. Missing SECRET_KEY => security warnings or startup failure. Misconfigured STATIC_ROOT => static serving broken in production.

Common beginner mistakes

- Committing SECRET_KEY to VCS.
- Having different settings in dev and prod without clear environment override pattern.
- Putting heavy logic in settings (avoid side effects).

urls.py (project-level)

What is it, literally?

- Root URLConf mapping URL patterns to views or including app url modules.

Example:

```
```python
```

```
from django.urls import path, include from django.contrib import admin

urlpatterns = [path('admin/', admin.site.urls), path('', include('finance.urls', namespace='finance')),
path('reports/', include('reports.urls', namespace='reports')),

]

```
```

Why does Django need it?

- The URL resolver traverses this list to locate the correct view for incoming requests.

How does it work internally?

- Django builds a URL resolver tree from urlpatterns at startup. For each incoming request path it tries patterns in order; when include() is hit, it delegates resolution to the included module with the remaining path.

When is it loaded/used?

- Loaded at startup (imported from ROOT_URLCONF). Resolution happens per-request.

What breaks if missing/wrong?

- Wrong import path -> ImportError at startup. Missing route -> 404. Misordered routes can shadow intended patterns.

Common beginner mistakes

- Not namespacing included urlconfs (leading to reverse() name collisions).
- Putting catch-all route at top, causing later patterns unreachable.

urls.py (app-level)

What is it, literally?

- Per-app url patterns that keep routing modular. Example finance/urls.py:

```
```python
```

```
from django.urls import path from . import views

app_name = 'finance' urlpatterns = [path("", views.dashboard, name='dashboard'), path('transactions/',
views.transaction_list, name='transaction_list'), path('transactions/int:pk/', views.transaction_detail,
name='transaction_detail'),

]
```

...

Why does Django need it?

- Separates concerns and keeps routing for each app localized.

How does it work internally?

- Included into project-level urls; when delegated, resolver looks into this list as if it's the top-level patterns.

When loaded/used?

- Imported at startup when project-level urls include it; pattern matching per-request.

What breaks if missing/wrong?

- Mistyped view name or module path => ImportError at startup or 404 at request.

Common beginner mistakes

- Forgetting to set app\_name causing reverse namespacing surprises.

views.py

-----

What is it, literally?

- Python module with view callables (function-based or class-based). They accept HttpRequest and return HttpResponse/JsonResponse, use forms/serializers and ORM. Example FBV:

```
```python
```

```
from django.shortcuts import render, get_object_or_404
```

```
from .models import Transaction
```

```
def transaction_list(request): qs = Transaction.objects.filter(user=request.user).select_related('category')[:50]
return render(request, 'finance/transaction_list.html', {'transactions': qs})
```

...

Example CBV:

```
```python
```

```
from django.views.generic import ListView class TransactionListView(ListView): model = Transaction
template_name = 'finance/transaction_list.html' paginate_by = 50 def get_queryset(self): return
Transaction.objects.filter(user=self.request.user).select_related('category')
```

...

## Why does Django need it?

- Views implement the application's request/response logic — they tie URL -> business logic -> presentation.

## How does it work internally?

- URL resolver returns view callable; for CBV, `as_view()` returns a function that constructs a view instance and calls `dispatch()`. The view receives `HttpRequest`, uses ORM/forms, and returns `HttpResponse`.

## When is it loaded/used?

- Views are imported when `URLConf` is imported at startup (so syntax/runtime errors in views can break startup), and executed per-request.

## What breaks if missing/wrong?

- Syntax error in `views.py` -> `ImportError` at startup. Returning wrong object type (not `HttpResponse`) -> `TypeError` at runtime. Unhandled exceptions propagate to 500 unless middleware handles.

## Common beginner mistakes

- Putting heavy business logic in view functions rather than service layer; forgetting to restrict data by `user` causing data leaks.

## models.py

-----

## What is it, literally?

- Python classes inheriting from `django.db.models.Model` declaring fields/relationships, Meta options, managers, methods.

## Example:

```
```python
```

```
from django.db import models class Transaction(models.Model):
```

```
    user = models.ForeignKey(settings.AUTH_USER_MODEL, on_delete=models.CASCADE,
related_name='transactions') amount = models.DecimalField(max_digits=12, decimal_places=2) date =
models.DateField() category = models.ForeignKey('Category', on_delete=models.PROTECT) note =
models.TextField(blank=True)
```

```
class Meta: indexes = [models.Index(fields=['user', 'date'])]
```

```
```
```

## Why does Django need it?

- Models are the canonical data schema for ORM mapping; they drive migrations and admin.

How does it work internally?

- Model class creation runs through `ModelBase` metaclass that collects fields and builds model's `_meta`. At migrate time Django uses this metadata to create/alter DB schema.

When is it loaded/used?

- Imported at startup when app registry loads; queries executed at runtime per-request.

What breaks if missing/wrong?

- Misspell field names => `AttributeError` at access. Incorrect `on_delete` choices cause cascade issues. Missing migrations if you change models without running `makemigrations` -> runtime DB mismatch.

Common beginner mistakes

- Relying only on Python-level validation and not enforcing DB-level constraints (i.e., leaving `NOT NULL` when you need it).

`admin.py`

-----

What is it, literally?

- Module registering models with Django admin using `ModelAdmin` classes for customization.

Example:

```
```python
```

```
from django.contrib import admin from .models import Transaction, Category
```

```
@admin.register(Transaction) class TransactionAdmin(admin.ModelAdmin): list_display = ('id', 'user', 'amount', 'date', 'category') list_filter = ('category', 'date') search_fields = ('note',)
```

```
```
```

Why does Django need it?

- Provides an out-of-the-box admin UI for CRUD on registered models.

How does it work internally?

- `admin.site` registers model options and dynamically generates `ModelAdmin` views/forms; admin URLs are added to project urls with `admin.site.urls`.

When is it loaded/used?

- `admin.py` is imported when `django.contrib.admin` is initialized (startup) if `INSTALLED_APPS` includes the app and the admin is enabled.

What breaks if missing/wrong?

- Bad admin configuration can raise exceptions at startup or when accessing admin pages.

### Common beginner mistakes

- Leaving admin open to public without proper `ALLOWED_HOSTS` or admin hardening.

apps.py

-----

What is it, literally?

- AppConfig class that configures app name and ready() hook.

### Example:

```
```python
```

```
from django.apps import AppConfig
class FinanceConfig(AppConfig):
    name = 'finance'
    verbose_name = 'Finance'
```

```
def ready(self):
    import finance.signals
    # register signal handlers safely
```

```
```
```

Why does Django need it?

- Registers app metadata in the app registry and provides startup hook to connect signals or perform app-level initialization.

How does it work internally?

- `django.setup()` iterates `INSTALLED_APPS`, imports AppConfig classes, and calls `ready()` after all apps loaded.

When is it loaded/used?

- At process startup during `django.setup()`.

What breaks if missing/wrong?

- Errors in `ready()` (heavy DB calls or import cycles) cause startup failure.

### Common beginner mistakes

- Executing DB queries in `ready()` (before DB is ready / in management commands) or causing circular imports.

middleware.py (or custom middleware classes)

-----

What is it, literally?

- Middleware are callables/classes placed in MIDDLEWARE list; they can inspect/modify requests/responses. Example custom middleware:

```
```python
```

```
class RequestTimerMiddleware:
    def init(self, get_response):
        self.get_response = get_response
    def call(self, request):
        start = time.time()
        response = self.get_response(request)
        duration = time.time() - start
        response['X-Elapsed'] = str(duration)
        return response
```

```
```
```

Why does Django need it?

- Centralized cross-cutting concerns (security, sessions, authentication, logging, CORS, etc.).

How does it work internally?

- Django wraps the view callable with middleware chain: each middleware receives `get_response` and returns a callable. On request phase middleware code runs top-down; on the way back response phase runs bottom-up.

When is it loaded/used?

- Middleware classes are imported at startup, and their **init** called; **call** executed per-request.

What breaks if missing/wrong?

- Misordered middleware can break auth, sessions, or CSRF. Heavy middleware can significantly increase request latency.

Common beginner mistakes

- Writing blocking I/O or expensive operations in middleware and causing performance degradation.

forms.py

```

```

What is it, literally?

- Modules defining `django.forms.Form` or `ModelForm` to validate and clean input.

Example:

```
```python
```

```
from django import forms
from .models import Transaction
```

```
class TransactionForm(forms.ModelForm):
    class Meta:
        model = Transaction
        fields = ['amount', 'date', 'category', 'note']
    def clean_amount(self):
        amt = self.cleaned_data['amount']
        if amt == 0:
            raise forms.ValidationError("Amount cannot be zero")
        return amt
```

```
```
```

Why does Django need it?

- Centralizes input validation and conversion for HTML forms.

How does it work internally?

- When view receives POST, instantiate form with POST data and call `form.is_valid()`; form runs field validators and `clean()` chain, populating `cleaned_data`.

When is it loaded/used?

- Imported in views and used per-request at form submission time.

What breaks if missing/wrong?

- Invalid forms/validation errors result in `form.is_valid()` False; if you forget to check, you might save invalid data.

Common beginner mistakes

- Trusting `form.is_valid()` implicitly without calling it; forgetting to call `form.save()` with `commit=False` when you need to set extra fields.

wsgi.py

-----

What is it, literally?

- Module exposing WSGI application callable used by WSGI servers.

Example:

```
```python
```

```
import os from django.core.wsgi import get_wsgi_application
os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'finance_tracker.settings') application =
get_wsgi_application()
```

```
```
```

Why does Django need it?

- It gives a standard WSGI callable to run Django in production (Gunicorn, uWSGI).

How does it work internally?

- `get_wsgi_application` returns a WSGI app that wraps Django request handling; the server calls `application(environ, start_response)`.

When is it loaded/used?

- At process start by WSGI server.



What breaks if missing/wrong?

- Wrong settings module or errors in this file prevent the WSGI server from starting.

Common beginner mistakes

- Running wsgi app directly in dev instead of runserver (lack of autoreload), or mismatching ASGI/WSGI expectations.

asgi.py

-----

What is it, literally?

- Module exposing ASGI application callable for async servers (Uvicorn, Daphne).

Example:

```
```python
```

```
import os from django.core.asgi import get_asgi_application
os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'finance_tracker.settings') application =
get_asgi_application()
```

```
```
```

Why does Django need it?

- Supports async views, websockets and long-lived connections when using ASGI servers and channels.

How does it work internally?

- ASGI scope and events passed to application; get\_asgi\_application returns a callable compatible with ASGI standard.

When is it loaded/used?

- At process start by ASGI server.

What breaks if missing/wrong?

- Using async features but starting WSGI server -> errors or no websocket support.

Common beginner mistakes

- Blocking code inside async views (e.g., synchronous DB or external calls) without running them in thread pool.

templates/ (folder)

-----

What is it, literally?

- Directory tree containing DTL files. Django loads them using template loaders configured in `TEMPLATES` setting. Example path: `templates/finance/transaction_list.html`

Why does Django need it?

- Provides view presentation layer for server-rendered pages.

How does it work internally?

- Template loaders find files; template engine parses into nodes; render compiles nodes, resolves variables using context processors + view context, returns a string.

When is it loaded/used?

- Loaded on demand per-request (or cached compiled template between requests if template caching enabled).

What breaks if missing/wrong?

- `TemplateDoesNotExist` when `render()` called or wrong context variables causing `KeyError` in templates if strict config set.

Common beginner mistakes

- Placing templates in app directory but not having app in `INSTALLED_APPS` or `TEMPLATE_DIRS` misconfigured.

static/ (folder)

-----

What is it, literally?

- CSS, JS, image assets stored under `static/` in apps or project-level static dirs.
- `STATICFILES_DIRS` lists dev directories; `STATIC_ROOT` is target for `collectstatic`. Why needed?
- Separates assets from templates and enables optimized serving in production (CDN or web server).

How it works internally?

- During development, static files served by `django.contrib.staticfiles`. In production, `collectstatic` gathers assets into `STATIC_ROOT` for Nginx/WhiteNoise/CloudFront.

When used?

- Per-request when template uses `{% static 'css/site.css' %}` tag to generate URL.

What breaks if missing/wrong?

- Missing static files leads to 404 and broken UI.

Common mistake

- Forgetting `collectstatic` on deploy or relying on runserver static serving in prod.

media/ (folder)

-----

What is it, literally?

- Directory where user-uploaded files are stored; configured by MEDIA\_ROOT and served at

## MEDIA\_URL.

Why needed?

- Stores files (uploads) separate from static assets and code.

How it works internally?

- FileField/ImageField stores path in DB; uploaded files saved to storage backend (local disk or S3) at save time.

When used?

- At runtime when users upload; serve either by dev server in DEBUG or via web server/cloud storage in prod.

What breaks if missing/wrong?

- File upload error; missing MEDIA\_ROOT -> can't save files; wrong permissions -> write errors.

Common mistake

- Serving media via runserver in production or forgetting to configure object storage and signed URLs for privacy.

---

## PART 3 — DJANGO COMMANDS (detailed internals & effects)

---

For each command: what it does internally, when to run it, what changes on disk/DB, and typical errors if misused.

python manage.py runserver

-----

What it does internally:

- Calls the runserver management command which:
- Calls django.setup() (loads settings, installed apps, app registry, AppConfig.ready()).
- Sets up autoreload (watching Python source files).
- Starts a development WSGI-compatible server (simple, single process/threaded) or ASGI dev server for async if configured. What to run it for:

- Local dev and quick testing. What changes on disk/DB:
- None (just process executes). Note: some code triggered on startup may perform side effects (e.g., signal registration). Common errors:
- ImportError due to bad settings or missing env vars; port already in use; DEBUG-dependent code failing in production.

`python manage.py makemigrations`

-----

What it does internally:

- Compares current model state (in-app models.py and model \_meta) with the "migration graph" (migrations files present) to detect changes.
- Generates migration files in `app/migrations/auto.py` containing Operation objects (AddField, CreateModel, etc.)

What to run it for:

- After adding/changing models to create migration scripts. What changes on disk/DB:
- Creates new migration Python files on disk (no DB changes yet). Common errors:
- Model import errors preventing makemigrations; changes that can't be autodetected (complex operations) require manual edits.

`python manage.py migrate`

-----

What it does internally:

- Reads migration graph, finds unapplied migrations, runs their operations in order.
- Each migration operation executes SQL or Python (RunPython) against DB using `schema_editor`.
- Records applied migrations in `django_migrations` table (applied migrations list). What to run it for:
- Apply schema and data migrations to the DB when deploying or developing after makemigrations. What changes on disk/DB:
- Alters DB schema (CREATE TABLE, ALTER TABLE, CREATE INDEX) and inserts migration records in `django_migrations`. Common errors:
- DB permissions error, locked tables, incompatible state (migrations out of sync), apply order conflicts.

`python manage.py showmigrations`

-----

What it does:

- Lists migrations for apps and marks which are applied (X) vs unapplied. When to run:
- Check migration state across environments. What it changes:

- Nothing.

Common issues:

- Migrations applied in DB but not present on disk or vice versa; divergence signals.

```
python manage.py sqlmigrate app_name migration_number
```

-----

What it does:

- Compiles a single migration's operations into SQL for the configured DB backend; prints it so you can preview. When to run:
- Inspect generated SQL, especially for large table changes. What it changes:
- Nothing on DB; pure preview. Common mistake:
- Relying on default SQL without reviewing impact on production (locks, table rewrites).

```
python manage.py createsuperuser
```

-----

What it does:

- Interactive prompt that creates an `auth.User` with `is_staff/is_superuser` flags; password hashed via configured password hasher. When to run:
- Initial setup for admin access in dev or staging. What it changes:
- Inserts rows in `auth_user` table. Common errors:
- Using default weak password policies; running in non-interactive CI needs `--noinput` with pre-seeded env variables.

```
python manage.py shell
```

-----

What it does:

- Bootstraps Django and drops into an interactive Python shell with Django configured; if `django-extensions` installed, `shell_plus` loads models automatically. When to run:
- Ad-hoc ORM testing/debugging, executing scripts with Django context. What changes:
- No changes unless you execute DB operations in shell. Common pitfalls:
- Running destructive commands on production DB accidentally if connected.

```
python manage.py collectstatic
```

-----

What it does:

- Uses staticfiles finders (app static directories, `STATICFILES_DIRS`) to collect and copy static assets into `STATIC_ROOT` or a storage backend (S3).
- Optional storage backends can fingerprint/narrow files (`ManifestStaticFilesStorage`). When to run:
- Before serving static assets in production or when deploying. What changes:
- Copies files into `STATIC_ROOT`; may write manifest (e.g., `staticfiles.json`) when using `ManifestStaticFilesStorage`. Common errors:

- Missing STATIC\_ROOT or permissions error; broken manifest if storage expects hashed filenames but templates reference originals (use {% static %}).

python manage.py test

-----

What it does:

- Discovers tests per Django testing policy (unittest-based), creates test database(s), runs migrations (or uses serialized test DB), runs test suite, then tears down test DB. When to run:
- CI, pre-commit checks, local testing. What changes:
- Creates temporary test DB and applies migrations; no persistent changes when complete. Common pitfalls:
- Tests relying on local uncommitted data, slow tests due to heavy fixtures, using production DB tests accidentally.

python manage.py dumpdata

-----

What it does:

- Serializes DB rows into fixture format (JSON by default), respecting natural keys if configured. When to run:
- Backups, seed data exports. What changes:
- Writes fixture files to disk; no DB change. Common issues:
- Dumping large DB causing huge fixture files; exposing sensitive data.

python manage.py loaddata

-----

What it does:

- Reads fixture(s) and deserializes them into model instances, inserting or updating DB rows. When to run:
- Seeding dev data, loading fixtures in tests. What changes:
- Modifies DB. Common mistakes:
- Loading fixtures incompatible with current schema; duplicates or PK conflicts.

Additional admin/dev commands

-----

- `python manage.py dbshell` — opens DB shell using configured DB credentials (psql for

Postgres).

- `python manage.py check` — runs project checks for common misconfigurations.
  - `python manage.py showmigrations` — list migrations.
- 

## QUICK TROUBLESHOOTING CHEAT SHEET (commands + typical errors)

---

- `ImportError` at runserver -> usually bad settings or missing dependency.
  - `OperationalError: no such column` -> forgot to run migrate in running environment.
  - `TemplateDoesNotExist` -> wrong template path or missing app in `INSTALLED_APPS` or missing add of templates dir.
  - `ModuleNotFoundError` on startup -> incorrect import or `PYTHONPATH`.
  - `Permission denied` writing to `STATIC_ROOT/MEDIA_ROOT` -> fix filesystem perms or storage config.
  - Migrations applied in prod but not in repo -> ensure migrations committed to VCS.
- 

## NEXT STEPS / OFFER

---

I've finished generating PART 2 (Project Structure) and PART 3 (Commands) for Prompt 1, as you asked. If you want, I can now:

- Continue and generate Prompt 2 in full (Development Workflow examples + ORM Mastery) exactly in the code-first, end-to-end manner you requested (include the three hands-on features and migrations/SQL/tests).
- Or pause here and let you review your project against this output before I proceed.

Tell me: proceed to Prompt 2 now (generate the Development Workflow feature examples and the full ORM Mastery content in the interactive, code-first style), or would you like to check this output first?

Added. Here's the startapp entry formatted to match the rest of the handbook (concise, concrete, and focused):

```
python manage.py startapp
```

What is it, literally?

- A management command that scaffolds a new Django app package (a Python package directory with a small set of files) ready for you to implement models, views, urls, tests, admin, etc.
- Example command: `python manage.py startapp finance`

What does it do internally?

- Creates a new directory named after the app (here: finance).
- Writes a set of boilerplate Python files into that directory so the folder is a valid Python package and a recognizable Django app.
- It does not modify `settings.py`, `INSTALLED_APPS`, or any existing files — it only creates files on disk.

What folders/files are generated (exact typical output) `finance/` **init.py** # makes finance a Python package  
`admin.py` # register models in Django admin `apps.py` # AppConfig subclass for app metadata & `ready()` hook  
`migrations/` # migration package **init.py** # marks migrations folder as package (empty initially) `models.py` #  
place model classes here `tests.py` # test cases for this app `views.py` # view callables / CBVs for this app

Minimal content examples (what's inside by default)

- `apps.py` from `django.apps` import `AppConfig` class `FinanceConfig(AppConfig):`

`name = 'finance'`

- `models.py` from `django.db` import `models`

## empty scaffolding ready for your model classes

---

- `migrations/init.py`

## empty - migration system will add files here after makemigrations

---

When would you run it?

- When you are adding a new feature domain or bounded context and want modularity (e.g., transactions, reports, accounts).
- Typical workflow: create the app directory first, then add `models/views/templates`, add to `INSTALLED_APPS`, then `makemigrations/migrate`.

What changes on disk?

- A new directory with the files listed above appears in the project workspace. Nothing else is changed (no settings update, no DB changes) until you edit files and run management commands.

What you must do next (common follow-ups)

- Add the app to `INSTALLED_APPS` in `settings.py` (or use the AppConfig dotted path): `INSTALLED_APPS += ['finance.apps.FinanceConfig']`
- Without this, Django will not register models, admin, migrations, templates/static lookups for this app.
- Create models in `finance/models.py` and run: `python manage.py makemigrations finance python manage.py migrate`
- Add urls for the app (`finance/urls.py`) and include them in project-level `urls.py`: `path('finance/', include('finance.urls', namespace='finance'))`

When is the app code loaded/used?

- The created files are imported when Django loads the app registry (`django.setup()`) — typically at process startup (`runserver` / `gunicorn` / `manage.py` commands). But nothing in the DB or runtime changes until you add code and run migrations or use the app's views.

What breaks if you forget to do the follow-ups?



- If you don't add the app to `INSTALLED_APPS`:
- Model classes won't be registered; makemigrations may not detect model changes.
- Admin registration in `admin.py` won't show in the admin.
- app-level templates/static may not be found by the app-specific finders (depending on your `TEMPLATE/STATIC` configuration).
- If you create an app name that shadows a Python stdlib module or an existing package (e.g., naming app "email" or "json"), you'll get import errors at startup.

### Common beginner mistakes

- Forgetting to add the new app to `INSTALLED_APPS` (most common).
- Naming the app the same as a top-level package in your repo (causes import shadowing).
- Running `startapp` inside an unexpected directory (creating nested apps or wrong relative import paths).
- Assuming `startapp` runs `makemigrations/migrate` — it does not. You must create models and then run `makemigrations` and `migrate`.

### Quick checklist after `startapp`

- ☐ Add app to `INSTALLED_APPS`
- ☐ Create models and run `makemigrations + migrate`
- ☐ Add `app.urls` and include them in `project urls.py`
- ☐ Register any models in `admin.py` if admin UI wanted
- ☐ Add templates/static folders as needed (`templates/finance/`, `static/finance/`)

### Example developer sequence (concrete)

- Create app: `python manage.py startapp finance`
- Register in `settings.py`: `INSTALLED_APPS += ['finance.apps.FinanceConfig']`
- Add a model in `finance/models.py`, then: `python manage.py makemigrations finance python manage.py migrate`
- Create `finance/urls.py`, wire in `project urls.py`, implement views/templates.

That's the full, concrete `startapp` entry — added into the handbook content as requested.

-----  
-----

Understood — you asked to skip the predict/quizzing step and have me continue immediately. Below is PART 2 — ORM MASTERY, delivered as a practical, code-first handbook you can use against your finance tracker. I cover field types, managers vs QuerySets, lazy evaluation, filters/get/exclude, relationships, `select_related` vs `prefetch_related` (with N+1 demonstration and SQL), `annotate/aggregate`, `transactions/atomic`, and migrations internals. Each topic follows the pattern you requested: real minimal syntax first, then SQL shown, then why/when, performance note, and a common beginner mistake.

Note: when I show SQL, I use PostgreSQL style examples (psycopg2) since your project likely uses Postgres in production. For your local SQLite behavior the SQL differs in dialect but the conceptual mapping is the same.

---

## PART 2 — ORM MASTERY

---

### SECTION A — FIELD TYPES (code first)

For each type: minimal model snippet, the typical SQL DDL it results in, why and when to use it, performance notes, common mistake.

#### 1. CharField

Code:

```
```python
```

```
from django.db import models
```

```
class Category(models.Model): name = models.CharField(max_length=100)
```

```
```
```

SQL (Postgres):

```
```sql
```

```
CREATE TABLE finance_category ( id serial PRIMARY KEY, name varchar(100) NOT NULL
```

```
);
```

```
```
```

Why/When: short textual values with known max length (names, slugs). Use `max_length` to enforce DB size. Performance: `varchar` uses less space than `text` for short fields; indexable. Common mistake: using `CharField` for unbounded text — prefer `TextField` for long descriptions.

#### 2. TextField

Code:

```
```python
```

```
class Note(models.Model): text = models.TextField(blank=True)
```

```
```
```

SQL:

```
```sql
```

```
text text;
```

...

Why/When: large free-form text (comments, descriptions). Performance: not suitable for indexing (except Gin trigram/full-text). Avoid scanning large fields unnecessarily. Common mistake: indexing TextField without appropriate full-text index.

3. IntegerField / PositiveIntegerField

Code:

```
```python
```

```
class Budget(models.Model): limit = models.IntegerField() month = models.PositiveSmallIntegerField()
```

...

SQL:

```
```sql
```

```
limit integer NOT NULL; month smallint NOT NULL;
```

...

Why/When: integers, counters, enums.

Performance: efficient, small footprint. Common mistake: using IntegerField for money (use DecimalField).

4. DecimalField

Code:

```
```python
```

```
from decimal import Decimal
```

```
class Transaction(models.Model): amount = models.DecimalField(max_digits=12, decimal_places=2)
```

...

SQL:

```
```sql
```

```
amount numeric(12,2) NOT NULL;
```

...

Why/When: monetary values requiring exactness. Performance: slower than integers; consider storing minor units as integer (cents) for extreme perf. Common mistake: using FloatField for money (precision issues).

5. DateField / DateTimeField

Code:

```
```python
```

```
class Transaction(models.Model): date = models.DateField() created_at =
models.DateTimeField(auto_now_add=True)
```

```
```
```

SQL:

```
```sql
```

```
date date NOT NULL; created_at timestamp with time zone NOT NULL;
```

```
```
```

Why/When: dates and timestamps; use timezone-aware datetime in Django settings (USE_TZ=True). Common mistake: naive datetimes; mixing timezone-aware/naive.

6. BooleanField / NullBooleanField

Code:

```
```python
```

```
is_recurring = models.BooleanField(default=False)
```

```
```
```

SQL:

```
```sql
```

```
is_recurring boolean NOT NULL DEFAULT false;
```

```
```
```

Why/When: flags. Common mistake: using NULLable booleans when three-state not desired.

7. ForeignKey

Code:

```
```python
```

```
class Transaction(models.Model): category = models.ForeignKey('Category', on_delete=models.PROTECT,
related_name='transactions')
```

```
```
```

SQL (FK constraint truncated):

```
```sql
```

```
category_id integer NOT NULL REFERENCES finance_category(id) ON DELETE
```

```
PROTECT;
```

```
```
```

Why/When: many-to-one relations (transaction -> category). Performance: joins; use `select_related` when retrieving related objects.

8. OneToOneField

Code:

```
```python
```

```
class Profile(models.Model): user = models.OneToOneField('auth.User', on_delete=models.CASCADE) avatar =
models.ImageField(upload_to='avatars/')
```

```
```
```

SQL:

```
```sql
```

```
user_id integer UNIQUE REFERENCES auth_user(id) ON DELETE CASCADE;
```

```
```
```

Why/When: extend user model with single-row profile. Common mistake: using `OneToOne` where FK uniqueness is not necessary.

9. ManyToManyField

Code:

```
```python
```

```
class Tag(models.Model): name = models.CharField(max_length=50)
```

```
class Transaction(models.Model): tags = models.ManyToManyField(Tag, related_name='transactions',
blank=True)
```

```
```
```

SQL: creates intermediary table `finance_transaction_tags(transaction_id, tag_id)` with indexes. Why/When: many-to-many relationships (transactions tagged with many tags). Performance: queries may require separate queries; use `prefetch_related`.

10. JSONField (Postgres)

Code:

```
```python
from django.contrib.postgres.fields import JSONField

class Event(models.Model): metadata = JSONField(default=dict)

```
```

SQL:

```
```sql
metadata jsonb NOT NULL DEFAULT '{}::jsonb;

```
```

Why/When: semi-structured data, flexible attributes. Performance: good but avoid storing frequently queried scalars inside JSON when indexing is required. Common mistake: overusing JSONField instead of normalized columns.

11. FileField / ImageField

Code:

```
```python
from django.db import models

class Receipt(models.Model): file = models.FileField(upload_to='receipts/%Y/%m/') image =
models.ImageField(upload_to='receipts/%Y/%m/', null=True, blank=True)

```
```

Why/When: user-uploaded files. Requires Pillow for ImageField. Common mistake: storing files in DB — Django stores file path in DB, files on disk or object storage.

SECTION B — MANAGERS vs QUERYSETS (code-first)

Code — custom manager:

```
```python

class TransactionQuerySet(models.QuerySet): def expenses(self): return self.filter(amount__lt=0)

class TransactionManager(models.Manager): def get_queryset(self): return TransactionQuerySet(self.model,
using=self._db) def recent(self): return self.get_queryset().order_by('-date')[:10]

```
```

```
class Transaction(models.Model): amount = models.DecimalField(max_digits=12, decimal_places=2) objects =
TransactionManager()

...

```

How it works: Manager is the entry point (`Model.objects`). It returns QuerySets. QuerySet methods chain lazily and are composable. You can call `Transaction.objects.recent()`.

Why: managers provide high-level APIs; QuerySets are chainable query builders.

Performance note: avoid calling methods that evaluate queries prematurely (e.g., `list(qs)`).

Common mistake: defining instance methods on Manager that return non-QuerySet results (harder to chain). Prefer QuerySet methods.

SECTION C — LAZY EVALUATION (proof & code)

Example code:

```
```python
qs = Transaction.objects.filter(user=user) # NO SQL yet print(type(qs)) # QuerySet

```

## SQL executed at iteration:

---

```
for tx in qs[:10]: print(tx.id) # triggers SQL

```

```
...

```

Proof (inspect SQL without executing):

```
```python
qs = Transaction.objects.filter(user=user, amount__lt=0) print(str(qs.query))

```

or in Django 3.2+: `print(qs.query.str())`

```
...

```

The printed SQL:

```
```sql
SELECT "finance_transaction"."id", "finance_transaction"."amount", ... FROM "finance_transaction" WHERE
"finance_transaction"."user_id" = 5 AND "finance_transaction"."amount" < 0 ORDER BY

```

```
"finance_transaction"."date" DESC;
```

```
```
```

Demonstration of laziness: constructing `qs` does not touch DB; methods like `.count()`, `list(qs)`, `.exists()`, iterating, slicing (that evaluates) will run SQL.

Why/When: lazy behavior allows composing queries efficiently.

Performance note: calling `len(qs)` forces evaluation — use `qs.count()` when you need DB count (but `count()` executes SQL COUNT which may be faster than fetching rows).

Common mistake: calling `list(qs)` in template or code unnecessarily causing large memory usage.

SECTION D — `filter()`, `exclude()`, `get()`, `values()`, `values_list()`

Examples:

```
```python
```

```
Transaction.objects.filter(user=user, amount__gt=0).order_by('-date')[:20]
```

```
Transaction.objects.exclude(status='deleted')
```

```
Transaction.objects.get(pk=5)
```

```
Transaction.objects.values('category__name').annotate(total=Sum('amount')) Transaction.objects.values_list('id', flat=True)
```

```
```
```

SQL examples:

- filter:

```
```sql
```

```
SELECT ... FROM finance_transaction WHERE user_id = 5 AND amount > 0 ORDER BY date DESC LIMIT 20;
```

```
```
```

- get:

```
```sql
```

```
SELECT ... FROM finance_transaction WHERE id = 5 LIMIT 1;
```

```
```
```


get raises `Transaction.DoesNotExist` or `MultipleObjectsReturned` if not unique.

Why/When: filter returns QuerySet (0..n rows), get returns single model instance.

Performance note: `values()` returns dictionaries and avoids model instantiation; faster when you only need specific columns.

Common mistake: using `get()` when multiple rows expected or not catching exceptions.

SECTION E — RELATIONSHIPS: ForeignKey / OneToOne / ManyToMany (with related_name)

Code:

```
```python
```

```
class UserProfile(models.Model): user = models.OneToOneField(settings.AUTH_USER_MODEL,
on_delete=models.CASCADE, related_name='profile')
```

```
class Transaction(models.Model): user = models.ForeignKey(settings.AUTH_USER_MODEL,
on_delete=models.CASCADE, related_name='transactions') categories = models.ManyToManyField('Category',
related_name='transactions', blank=True)
```

```
```
```

Access patterns:

```
```python
```

## forward FK

---

`tx.user` # simple attribute (no extra query if `select_related` used)

## reverse FK

---

`user.transactions.all()` # QuerySet, hits DB when evaluated

## one-to-one

---

`user.profile` # cached after access

## many-to-many

---

`tx.categories.all()` # separate query or `prefetch_related` necessary

```

SQL notes:

- FK uses a `category_id` column and a constraint.
- M2M creates an intermediary table (`finance_transaction_categories`).

Why/When:

- FK: many items belong to one parent.
- OneToOne: single extension.
- M2M: multiple tags/categories.

Performance note: reverse relations require extra queries per object unless prefetched.

Common mistake: not specifying `related_name` and later writing `user.transaction_set` which is ok, but `related_name` is clearer and avoids conflicts.

SECTION F — N+1 PROBLEM, `select_related` vs `prefetch_related` (code, SQL, fix)

Scenario: list transactions and show category name for each — naive version:

```python

```
qs = Transaction.objects.filter(user=user)[:50]
for tx in qs:
 print(tx.category.name)
For each tx, ORM will perform a query for category if not cached
```

```

This causes N+1: 1 query for transactions + N queries (one per transaction) for categories.

Demonstration SQLs:

- Initial transaction query:

```sql

```
SELECT id, amount, category_id FROM finance_transaction WHERE user_id=5 LIMIT 50;
```

```

- For each tx, category query:

```sql

```
SELECT id, name FROM finance_category WHERE id = <category_id>;
```

```

If 50 transactions -> 51 queries.

Fix using `select_related` (works for FK, OneToOne):

```
```python
```

```
qs = Transaction.objects.filter(user=user).select_related('category')[:50] for tx in qs: print(tx.category.name)
```

```
```
```

Now SQL:

```
```sql
```

```
SELECT t.id, t.amount, t.category_id, c.id, c.name FROM finance_transaction t LEFT JOIN finance_category c ON t.category_id = c.id WHERE t.user_id = 5
```

```
LIMIT 50;
```

```
```
```

All in one query — no extra per-row queries.

`prefetch_related` (for M2M and reverse FK) issues:

Example: you want transactions and their tags (many-to-many). Using `prefetch_related`:

```
```python
```

```
qs = Transaction.objects.filter(user=user).prefetch_related('tags') for tx in qs: tags = list(tx.tags.all()) # no extra queries
```

```
```
```

SQL executed:

- Main transaction query
- Second query to fetch tags for all transactions using transaction ids, then ORM assigns tags to each transaction in Python memory.

Why/When:

- `select_related`: use for single-row joinable relations (FK, OneToOne) when you want related object in same query.
- `prefetch_related`: use for M2M or reverse FK; it runs additional query but avoids per-object queries.

Performance note:

- `select_related` increases row width; fine for small joins.
- `prefetch_related` issues: can pull large data into memory; use only for needed fields.

Common mistake: using `select_related` for many-to-many (doesn't work) or using both indiscriminately; or prefetching huge resultsets and causing memory spikes.

SECTION G — `annotate()` / `aggregate()` (code & SQL)

Code examples:

1. Aggregate total amounts for user:

```
```python
```

```
from django.db.models import Sum
```

```
total = Transaction.objects.filter(user=user).aggregate(total=Sum('amount'))
```

```
total = {'total': Decimal('12345.67')}
```

---

```
```
```

SQL:

```
```sql
```

```
SELECT SUM(amount) AS total FROM finance_transaction WHERE user_id = 5;
```

```
```
```

2. Annotate per-category totals:

```
```python
```

```
from django.db.models import Sum
```

```
qs = Transaction.objects.values('category__name').annotate(total=Sum('amount')).order_by('-total
```

```
')
```

```
```
```

SQL:

```
```sql
```

```
SELECT c.name AS category__name, SUM(t.amount) AS total FROM finance_transaction t JOIN
finance_category c ON t.category_id = c.id WHERE t.user_id = 5 GROUP BY c.name ORDER BY total DESC;
```

```
```
```

Why/When: use aggregate for single-value summary, annotate to enrich rows with computed values.

Performance note: GROUP BY can be expensive on large tables; ensure proper indexes (e.g., on category_id) and consider pre-aggregated materialized views for heavy analytics.

Common mistake: iterating over annotate results and for each performing another DB query — combine into single annotated query.

SECTION H — Transactions and atomic()

Code:

```
```python
```

```
from django.db import transaction
```

```
def transfer_funds(from_user, to_user, amount):
```

```
 with transaction.atomic():
 from_acc = Account.objects.select_for_update().get(user=from_user)
 to_acc = Account.objects.select_for_update().get(user=to_user)
 from_acc.balance -= amount
 to_acc.balance += amount
 from_acc.save()
 to_acc.save()
```

```
```
```

What it does internally:

- `atomic()` opens a DB transaction. If any exception occurs inside, transaction rolls back.
- `select_for_update()` acquires row-level locks to prevent concurrent modifications.

SQL example sequence (Postgres):

```
```sql
```

```
BEGIN;
```

```
SELECT ... FROM account WHERE user_id = X FOR UPDATE;
SELECT ... FROM account WHERE user_id = Y FOR UPDATE;
UPDATE account SET balance = balance - 100 WHERE id = ...;
UPDATE account SET balance = balance + 100 WHERE id = ...;
```

```
COMMIT;
```

```
```
```

Why/When: use for multi-step DB changes that must be atomic (payments, transfers, inventory decrement).

Performance note: long transactions reduce concurrency and increase locking contention; keep transactions short.

Common mistake: performing network calls inside atomic block (slow) — do external calls before transaction or after commit hooks.

SECTION I — MIGRATIONS INTERNALS (what is inside migration file)

A migration file (auto-generated) looks like:

```
```python
```

```
finance/migrations/0003_add_field.py
```

---

```
from django.db import migrations, models
```

```
class Migration(migrations.Migration): dependencies = [
```

```
('finance', '0002_auto_20260601_1234'),
```

```
]
```

```
operations = [migrations.AddField(model_name='transaction', name='note',
```

```
field=models.TextField(blank=True),
```

```
),
```

```
migrations.AlterField(...), migrations.RunPython(forward_func, backward_func), # data migration example
```

```
]
```

```
```
```

What's inside:

- dependencies: migration graph
- operations: list of schema or data operations (CreateModel, AddField, RemoveField, AlterField, RunPython, RunSQL, CreateIndex, RemoveIndex)

Schema vs Data migrations:

- Schema migration: AddField, CreateModel — changes structure.
- Data migration: RunPython with Python code to transform data — used to backfill new columns or migrate values.

Squashing:

- `python manage.py squashmigrations appname start_migration` merges multiple migration files into a single file to speed up initial migrate. Squashed migration still references historical migrations.

What happens on `makemigrations`:

- Django compares model state (models.py) vs recorded state in migration files to create operations.

What happens on `migrate`:

- Django computes unapplied operations, runs them in order, records applied migrations in `django_migrations` table.

Common mistakes:

- Editing migration files incorrectly after they've been applied in other environments (causes diverging state).
- Running `makemigrations` and not reviewing generated SQL for heavy operations (e.g., adding a NOT NULL column without default can lock table).

SECTION J — MIGRATIONS: data migrations example

Add a non-nullable column that needs backfill:

- Expand step: Add nullable column

```
```python
```

```
class Migration(migrations.Migration):
 operations = [
 migrations.AddField(
 model_name='budget',
 name='monthly_limit',
 field=models.DecimalField(
 null=True, max_digits=10, decimal_places=2),
),
]
```
```

Run migrate -> safe (no table rewrite usually).

- Backfill data via RunPython (data migration)

```
```python
```

```
def forwards(apps, schema_editor):
 Budget = apps.get_model('finance', 'Budget')
 for b in Budget.objects.filter(monthly_limit__isnull=True):
 b.monthly_limit = Decimal('1000.00')
 b.save()

 operations = [
 migrations.RunPython(forwards, reverse_code=migrations.RunPython.noop),
]
```
```

- Contract: set NOT NULL and remove null=True

```
```python
```

```
migrations.AlterField(model_name='budget', name='monthly_limit', field=models.DecimalField(null=False,
max_digits=10, decimal_places=2),
)
'''
```

Why this pattern: avoids table locks and allows smooth zero-downtime migrations.

Common mistake: trying to add NOT NULL column without default on large tables (blocks or fails).

## SECTION K — INDEXES & OPTIMIZATION (short practical guide)

Create index via model Meta:

```
```python
class Transaction(models.Model): user = models.ForeignKey(...) date = models.DateField()

class Meta: indexes = [ models.Index(fields=['user', 'date']),
]
'''
```

SQL generated:

```
```sql
CREATE INDEX finance_transaction_user_date_idx ON finance_transaction (user_id, date);
'''
```

When: index fields used in WHERE/JOIN/ORDER BY. Composite index ordering matters.

Explain EXPLAIN ANALYZE:

```
```python
qs = Transaction.objects.filter(user=user).order_by('-date')[:50] print(qs.explain()) # Django supports
QuerySet.explain()
'''
```

Use it to inspect plan (index scan vs sequential scan).

Common mistake: indexing low-cardinality columns (e.g., boolean) yields little benefit and costs write overhead.

SECTION L — QUERYSET METHODS QUICK REFERENCE (practical)

- `filter()` -> WHERE
- `exclude()` -> WHERE NOT
- `get()` -> fetch single row, raises if not unique
- `order_by()` -> ORDER BY
- `annotate()` -> add aggregated columns (GROUP BY)
- `aggregate()` -> overall aggregation
- `select_related()` -> SQL JOIN for FK/OneToOne
- `prefetch_related()` -> separate query + in-memory join for M2M/reverse
- `values()` -> dicts; `values_list()` -> tuples/flat
- `only()` / `defer()` -> limit loaded columns to reduce IO
- `exists()` -> fast test (SELECT 1 ...)
- `count()` -> SQL COUNT()
- `delete()` -> deletes rows; beware cascade
- `update()` -> database-level update without loading instances (fast but bypasses `model.save()`)

Performance notes:

- `update()` and `delete()` operate at DB level (no signals or `save()` executed).
- `bulk_create()` inserts many rows faster but bypasses `save()`.

Common mistakes:

- Calling `.count()` on an evaluated queryset (if you already fetched rows, `len(qs)` may be in memory).
 - Using `values()` then expecting model instances.
-

SECTION M — PRACTICAL TIPS & CHECKLIST (for feature work)

When adding or changing DB-backed feature:

- Implement model change + add `null=True` if needed.
 - Run `makemigrations` -> inspect migration file.
 - Run `sqlmigrate app 000X` to preview SQL (especially CREATE INDEX, ALTER TABLE).
 - Run `migrate` in dev.
 - Write tests (model + view).
 - Use `select_related` / `prefetch_related` in views/templates where you will access related objects.
 - Run `EXPLAIN ANALYZE` for slow queries (use `qs.explain()`).
-

PART 2 — WHAT YOU SHOULD BE ABLE TO DO AFTER THIS

- Read any model class and map each field to DB column and SQL type.
- Explain QuerySet lazy evaluation and prove it in shell using `str(qs.query)` and

`qs.explain()`.

- Identify an N+1 problem in a view/template and fix it with `select_related/prefetch_related`.
- Build an annotate/aggregate query (e.g., monthly totals per category) and explain the

GROUP BY SQL.

- Write `with transaction.atomic()` code with `select_for_update()` to prevent race conditions.
 - Inspect and reason about a migration file (operations list) and implement a safe expand/backfill/contract pattern.
-

If you want me to continue now with the remaining prompts (Templates/Static/Media + Auth

- Middleware, then DRF bridge and interview cheatsheet) I will proceed in the same code-first, step-by-step, practical style. You asked to cover everything in a sequence; tell me to continue and I will deliver the next part immediately. Understood — I'll generate the remaining parts of Prompt 2: the three complete, end-to-end Development-Workflow examples. Each example shows the exact file edits (with concrete code you can paste), the migration file content and example SQL, tests (Django test code), the commands to run, deployment/dev notes, and small explanations of why each change is where it is.

I'll assume your project is named `finance_tracker` and the app is `finance`. If your app names differ, replace `finance` with your app name. I'll also assume you use Django 3.2+/4.x and PostgreSQL in production but SQLite in local dev; SQL shown will be PostgreSQL-style.

Example 1 — Add a profile picture to User Summary: Add a one-to-one Profile model with ImageField (avatar). Register Profile in admin, add form + view to upload/change avatar, add template changes, add media settings, run migrations, and tests.

Files changed/created (exact):

- `finance/models.py` — add Profile model
- `finance/admin.py` — register Profile
- `finance/forms.py` — add ProfileForm (or AvatarForm)
- `finance/views.py` — add `profile_view` to show/edit avatar
- `finance/urls.py` — add url pattern for profile
- `templates/finance/profile.html` — add template for upload/view
- `settings.py` — `MEDIA_ROOT/MEDIA_URL` config (if not present)
- requirements (pip) — add Pillow if not already installed

- migrations — makemigrations creates finance/migrations/000X_profile.py
- tests in finance/tests.py — add test for avatar upload

Step-by-step code

1. models.py Append or add class Profile to finance/models.py:

```
```python
```

## finance/models.py

---

```
from django.conf import settings from django.db import models from django.dispatch import receiver import os
```

```
class Profile(models.Model): user = models.OneToOneField(settings.AUTH_USER_MODEL, on_delete=models.CASCADE, related_name='profile'
```

```
)
```

```
avatar = models.ImageField(upload_to='avatars/%Y/%m/%d/', null=True, blank=True)
```

```
def avatar_url(self): if self.avatar: return self.avatar.url
```

## fallback placeholder

---

```
return '/static/img/default-avatar.png'
```

```
def str(self): return f'Profile({self.user.username})'
```

## Optional: auto-create Profile for new users (signal)

---

```
from django.db.models.signals import post_save from django.dispatch import receiver
```

```
@receiver(post_save, sender=settings.AUTH_USER_MODEL) def create_user_profile(sender, instance, created, **kwargs): if created:
```

```
Profile.objects.create(user=instance)
```

```
```
```

Why OneToOne: we extend auth.User without swapping user model. This is safe and minimal.

2. admin.py

Register Profile in admin to view/edit avatars:

```
```python
```

## finance/admin.py

---

```
from django.contrib import admin from .models import Profile, Transaction, Category # existing models
```

```
@admin.register(Profile) class ProfileAdmin(admin.ModelAdmin): list_display = ('user',) search_fields = ('user__username', 'user__email')
```

```
'''
```

3. forms.py Create a simple ModelForm to upload/change avatar:

```
'''python
```

## finance/forms.py

---

```
from django import forms from .models import Profile
```

```
class ProfileForm(forms.ModelForm): class Meta: model = Profile fields = ['avatar']
```

```
'''
```

4. views.py Add view to display and update profile (FBV):

```
'''python
```

## finance/views.py

---

```
from django.contrib.auth.decorators import login_required from django.shortcuts import render, redirect from .forms import ProfileForm
```

```
@login_required
```

```
def profile_view(request): profile = getattr(request.user, 'profile', None) if request.method == 'POST': form = ProfileForm(request.POST, request.FILES, instance=profile) if form.is_valid(): form.save() return redirect('finance:profile') else: form = ProfileForm(instance=profile) return render(request, 'finance/profile.html', {'form': form, 'profile': profile})
```

```
'''
```

5. urls.py (app) Add route:

```
'''python
```

## finance/urls.py

---

```
from django.urls import path from . import views
```

```
app_name = 'finance'
```

```
urlpatterns = [path('profile/', views.profile_view, name='profile'),
```

## ... existing urls

---

```
]
```

```
'''
```

6. template: templates/finance/profile.html Create template to show and upload avatar:

```
```django
```

```
{% extends "base.html" %} {% load static %} {% block content %}
```

Your profile

```
 avatar ##
```

```
{% csrf_token %} {{ form.non_field_errors }} ##
```

```
{{ form.avatar.label_tag }}
```

```
{{ form.avatar }} ##
```

```
 ##
```

```
{% endblock %} ## '''
```

7. settings.py — media settings Ensure MEDIA_URL and MEDIA_ROOT exist:

```
```python
```

## settings.py

---

```
import os BASE_DIR = Path(file).resolve().parent.parent
```

```
MEDIA_URL = '/media/' MEDIA_ROOT = os.path.join(BASE_DIR, 'media')
```

```
'''
```

In development urls (project-level urls.py) add media serve:

```
```python
```

```
from django.conf import settings from django.conf.urls.static import static
```

```
urlpatterns = [
```

... your url patterns

```
]
if settings.DEBUG: urlpatterns += static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
'''
```

8. requirements Install Pillow for ImageField support:

```
```bash
pip install Pillow pip freeze > requirements.txt
'''
```

9. migrations

## Commands:

```
```bash
python manage.py makemigrations finance python manage.py migrate
'''
```

Example generated migration (finance/migrations/000X_create_profile.py) — simplified:

```
```python
from django.db import migrations, models import django.db.models.deletion import django.conf

class Migration(migrations.Migration): dependencies = [
 migrations.swappable_dependency(django.conf.settings.AUTH_USER_MODEL),

 ('finance', '0009_previous_migration'),

]

operations = [migrations.CreateModel(name='Profile', fields=[('id', models.AutoField(primary_key=True,
serialize=False, auto_created=True)), ('avatar', models.ImageField(upload_to='avatars/%Y/%m/%d/', null=True,
blank=True)), ('user', models.OneToOneField(on_delete=django.db.models.deletion.CASCADE,
related_name='profile', to=django.conf.settings.AUTH_USER_MODEL)),

],

),

]
'''
```

SQL preview (sqlmigrate):

```
```sql
```

```
-- example: finance 000X_create_profile CREATE TABLE finance_profile ( id serial PRIMARY KEY, avatar
varchar(...) NULL, user_id integer NOT NULL REFERENCES auth_user(id) ON DELETE CASCADE
```

```
);
```

```
CREATE UNIQUE INDEX finance_profile_user_id_uniq ON finance_profile(user_id);
```

```
```
```

10. tests Add tests for profile creation and upload. Use SimpleUploadedFile:

```
```python
```

finance/tests.py

```
from django.test import TestCase, Client from django.contrib.auth import get_user_model from
django.core.files.uploadedfile import SimpleUploadedFile from .models import Profile import io
```

```
User = get_user_model()
```

```
class ProfileTests(TestCase): def setUp(self): self.user = User.objects.create_user(username='test',
password='pass') self.client = Client() self.client.login(username='test', password='pass')
```

```
def test_profile_auto_created(self):
```

profile created by post_save signal

```
self.assertTrue(hasattr(self.user, 'profile')) self.assertIsInstance(self.user.profile, Profile)
```

```
def test_avatar_upload(self):
```

create a small in-memory file

```
img = SimpleUploadedFile('avatar.jpg', b'\xff\xd8\xff\xe0' + b'0'*1024, content_type='image/jpeg') resp =
self.client.post('/finance/profile/', {'avatar': img}, follow=True) self.user.refresh_from_db()
self.assertTrue(self.user.profile.avatar) # path set
```

file exists at MEDIA_ROOT

```
self.assertTrue(self.user.profile.avatar.path.endswith('avatar.jpg'))
```

```
```
```

Note: in CI or tests, ensure MEDIA\_ROOT is a temp dir and cleaned up (Django TestCase creates temp dirs when configured).

## 11. run &amp; verify

## Commands:

```
```bash
```

```
pip install Pillow python manage.py makemigrations finance python manage.py migrate python manage.py runserver
```

visit <http://127.0.0.1:8000/finance/profile/> while logged in

```
python manage.py test finance.tests.ProfileTests
```

```
```
```

## Notes and cautions

- Serving media in production: do NOT use runserver. Use Nginx, S3, or WhiteNoise (WhiteNoise not for media typically). Configure MEDIA\_ROOT to a persistent storage or use cloud storage backend (django-storages + S3).
- Validate uploaded files: file size limits and content-type checks to avoid malicious uploads. Use ImageField validators or custom clean().

Example 1 complete.

```

```

Example 2 — Add search/filter to transactions Summary: Add search and filters (by text q, category, date\_from/date\_to) to the transaction list view and template. Use GET params; implement in view using ORM filters and select\_related to avoid N+1. Optionally add index for ILIKE performance.

Files changed/created:

- finance/views.py — modify transaction\_list view to accept filters
- finance/templates/finance/transaction\_list.html — add search form and preserve query params in pagination links if any
- finance/urls.py — existing list URL used; no new migrations required
- treatments: optional migration to add trigram index for text search (Postgres) — finance/migrations/000X\_add\_trgm\_index.py
- tests: finance/tests.py — add tests for search and filters

Step-by-step code

1. views.py — enhanced transaction\_list

```
```python
```


finance/views.py

```
from django.shortcuts import render
from django.db.models import Q
from .models import Transaction, Category
```

```
def transaction_list(request):
    qs = Transaction.objects.filter(user=request.user).select_related('category')
    q = request.GET.get('q', '').strip()
    category_id = request.GET.get('category')
    date_from = request.GET.get('date_from')
    date_to = request.GET.get('date_to')
```

```
    if q:
```

simple text search on note or category name

```
    qs = qs.filter(
```

```
        Q(note__icontains=q) |
```

```
        Q(category__name__icontains=q)
```

```
    )
```

```
    if category_id:
        qs = qs.filter(category_id=category_id)
    if date_from:
        qs = qs.filter(date__gte=date_from)
    if date_to:
        qs = qs.filter(date__lte=date_to)
```

```
    qs = qs.order_by('-date') # final ordering
```

optional: pagination

```
from django.core.paginator import Paginator
paginator = Paginator(qs, 25)
page_number = request.GET.get('page')
page_obj = paginator.get_page(page_number)
```

```
context = {
    'transactions': page_obj,
    'q': q,
    'category_id': category_id,
    'date_from': date_from,
    'date_to': date_to,
}
```

```
return render(request, 'finance/transaction_list.html', context)
```

```
```
```

Key points:

- We call `select_related('category')` to avoid per-row category queries.
- Use `Q` for OR search.
- Use pagination to limit rows.

2. template — add search form snippet

```
```django
```

```
{# templates/finance/transaction_list.html #}
```

```
{{ q }}
```

All categories ▼

dd-mm-yyyy

dd-mm-yyyy

Search

```
##
```

```
{% for tx in transactions %}
```

- ```
{{ tx.date }} - {{ tx.amount }} - {{ tx.category.name }} - {{ tx.note }}
```
- ```
{% endfor %} ##
```

```
{# pagination links - keep existing querystring parameters #} {% if transactions.has_previous %} Previous {%
endif %} {% if transactions.has_next %} Next {% endif %}
```

```
...
```

Note: Using Category model in template directly is acceptable for small sets; for large lists, pass categories in context from view or use a context processor.

3. tests — add tests for search/filter

```
python
```

finance/tests.py

```
from django.test import TestCase, Client from django.contrib.auth import get_user_model from .models
import Transaction, Category from decimal import Decimal from datetime import date
```

```
User = get_user_model()
```

```
class TransactionFilterTests(TestCase):
    def setUp(self):
        self.user = User.objects.create_user(username='u',
        password='p')
        self.client = Client()
        self.client.login(username='u', password='p')
```

create categories

```
self.cat_food = Category.objects.create(name='Food')
```

```
self.cat_travel = Category.objects.create(name='Travel')
```

create transactions

```
Transaction.objects.create(user=self.user, amount=Decimal('10.00'), date=date(2026,6,1),
category=self.cat_food, note='Grocery store')
Transaction.objects.create(user=self.user,
amount=Decimal('50.00'), date=date(2026,6,2), category=self.cat_travel, note='Taxi to airport')
```

```
def test_search_by_note(self):
    resp = self.client.get('/finance/transactions/?q=Grocery')
    self.assertContains(resp, 'Grocery')
    self.assertNotContains(resp, 'Taxi')
```

```
def test_filter_by_category(self):
    resp = self.client.get(f'/finance/transactions/?category={self.cat_travel.id}')
    self.assertContains(resp, 'Taxi')
    self.assertNotContains(resp, 'Grocery')
```

```

#### 4. SQL examples (illustrative)

- Base list query with `select_related`:

```sql

```
SELECT t.id, t.amount, t.date, t.note, t.category_id, c.id, c.name FROM finance_transaction t LEFT JOIN
finance_category c ON t.category_id = c.id WHERE t.user_id = 5 ORDER BY t.date DESC
```

LIMIT 25 OFFSET 0;

```

- With `q` filter (case-insensitive `ILIKE` in Postgres):

```sql

```
WHERE (t.note ILIKE '%grocery%' OR c.name ILIKE '%grocery%') AND t.user_id = 5
```

```

#### 5. Performance considerations

- `ILIKE '%term%'` cannot use a standard b-tree index; consider:
- Creating trigram/Gin index on note for fast `ILIKE`:

```sql

```
CREATE EXTENSION IF NOT EXISTS pg_trgm; CREATE INDEX finance_tx_note_trgm_idx ON finance_transaction
USING gin (note gin_trgm_ops);
```

```

Migration for that is possible with RunSQL.

- Or use full-text search (`tsvector`) for more robust search.
- Always `select_related` for FK access and `prefetch_related` for M2M.
- Limit and paginate to avoid pulling too many rows.

#### 6. Optional migration to add trigram index (Postgres)

Migration file sample:

```python

```
from django.db import migrations
```

```
class Migration(migrations.Migration): dependencies = [
```

```
('finance', '0009_previous'),
```

```

]

operations = [ migrations.RunSQL( "CREATE EXTENSION IF NOT EXISTS pg_trgm;", reverse_sql="-- no-op"

),

migrations.RunSQL( "CREATE INDEX finance_transaction_note_trgm_idx ON finance_transaction USING gin
(note gin_trgm_ops);", reverse_sql="DROP INDEX IF EXISTS finance_transaction_note_trgm_idx;"

),

]

'''

```

Run:

```
```bash
```

```
python manage.py makemigrations finance # may not auto-detect RunSQL; you create file manually python
manage.py migrate
```

```
'''
```

7. run & verify No new models, so usually only runserver and tests:

```
```bash
```

```
python manage.py runserver python manage.py test finance.tests.TransactionFilterTests
```

```
'''
```

Example 2 complete.

```
---
```

Example 3 — Add a new Budget category with a monthly limit Interpretation: Add a Budget model that links user + category + monthly_limit; implement model, migration, admin, form, view, and template to create a budget for a category.

Files changed/created:

- finance/models.py — add Budget model
- finance/migrations/000X_create_budget.py — migration created by makemigrations
- finance/admin.py — register Budget
- finance/forms.py — BudgetForm
- finance/views.py — add budget_create_view and budget_list_view

- `finance/urls.py` — add routes
- `templates/finance/budget_form.html`, `budget_list.html` — templates
- tests — add tests for budget creation and uniqueness

Step-by-step code

1. `models.py` — Budget model

```
```python
```

## finance/models.py

---

```
from django.conf import settings
from django.db import models
from decimal import Decimal

class Budget(models.Model):
 user = models.ForeignKey(settings.AUTH_USER_MODEL,
 on_delete=models.CASCADE, related_name='budgets')
 category = models.ForeignKey('Category',
 on_delete=models.PROTECT, related_name='budgets')
 monthly_limit = models.DecimalField(max_digits=12, decimal_places=2)
 created_at = models.DateTimeField(auto_now_add=True)

 class Meta:
 unique_together = ('user', 'category') # one budget per user per category
 indexes = [models.Index(fields=['user', 'category'])]

 def __str__(self):
 return f'{self.user.username} - {self.category.name} limit {self.monthly_limit}'

 ...
```

Why `unique_together`: prevent duplicate budgets for same user/category.

2. `admin.py` — register Budget

```
```python
```

finance/admin.py

```
from .models import Budget

@admin.register(Budget)
class BudgetAdmin(admin.ModelAdmin):
    list_display = ('user', 'category', 'monthly_limit')
    list_filter = ('category',)
    search_fields = ('user__username',)

    ...
```

3. `forms.py` — BudgetForm

```
```python
```

## finance/forms.py

---

```
from django import forms
from .models import Budget
```

```
class BudgetForm(forms.ModelForm):
 class Meta:
 model = Budget
 fields = ['category', 'monthly_limit']
```

```
...
```

4. views.py — create/list budgets

```
```python
```

finance/views.py

```
from django.contrib.auth.decorators import login_required
from django.shortcuts import render, redirect, get_object_or_404
from .forms import BudgetForm
from .models import Budget
```

@login_required

```
def budget_list(request):
    budgets = Budget.objects.filter(user=request.user).select_related('category')
    return render(request, 'finance/budget_list.html', {'budgets': budgets})
```

@login_required

```
def budget_create(request):
    if request.method == 'POST':
        form = BudgetForm(request.POST)
        if form.is_valid():
```

create with user

```
        budget = form.save(commit=False)
        budget.user = request.user
        budget.save()
        return redirect('finance:budget_list')
    else:
        form = BudgetForm()
        return render(request, 'finance/budget_form.html', {'form': form})
```

```
...
```

5. urls.py — add routes

```
```python
```

## finance/urls.py

---

```
urlpatterns += [
 path('budgets/', views.budget_list, name='budget_list'),
 path('budgets/create/', views.budget_create, name='budget_create'),
```

```
]
```

```
...
```

## 6. templates — budget\_form.html

``django

```
{# templates/finance/budget_form.html #} {% extends "base.html" %} {% block content %}
```

## Create Budget

---

```
{% csrf_token %} {{ form.as_p }} ##
{% endblock %} ## ``
```

## 7. templates — budget\_list.html

``django

```
{# templates/finance/budget_list.html #} {% extends "base.html" %} {% block content %}
```

## Your Budgets

---

##

```
{% for b in budgets %}
• {{ b.category.name }} — {{ b.monthly_limit }}
 {% empty %}
• No budgets yet
{% endfor %} ##
```

```
Create Budget {% endblock %} ## ``
```

## 8. migrations

## Commands:

``bash

```
python manage.py makemigrations finance python manage.py migrate
```

``

Example migration file (auto-generated):

``python

```
class Migration(migrations.Migration): dependencies = [('finance', '000X_previous'),
```

]

```
operations = [migrations.CreateModel(name='Budget', fields=[('id', models.AutoField(primary_key=True, ...)),
('monthly_limit', models.DecimalField(max_digits=12, decimal_places=2)), ('created_at',
```

```
models.DateTimeField(auto_now_add=True)),

('user', models.ForeignKey(on_delete=..., to=settings.AUTH_USER_MODEL)), ('category',
models.ForeignKey(on_delete=..., to='finance.Category')),

],

options={'unique_together': (('user', 'category'))},

),

migrations.AddIndex(model_name='budget', index=models.Index(fields=['user', 'category'],
name='finance_budget_user_category_idx'),

),

]

'''
```

SQL preview (sqlmigrate):

```
```sql
```

```
CREATE TABLE finance_budget ( id serial PRIMARY KEY, monthly_limit numeric(12,2) NOT NULL, created_at
timestamp with time zone NOT NULL, user_id integer NOT NULL REFERENCES auth_user(id) ON DELETE
CASCADE, category_id integer NOT NULL REFERENCES finance_category(id) ON DELETE
```

```
CASCADE
```

```
);

CREATE UNIQUE INDEX finance_budget_user_category ON finance_budget (user_id, category_id); CREATE
INDEX finance_budget_user_category_idx ON finance_budget (user_id, category_id);

'''
```

9. tests — create budget test

```
```python
```

## finance/tests.py

---

```
class BudgetTests(TestCase):
 def setUp(self):
 self.user = User.objects.create_user(username='b', password='p')
 self.client = Client()
 self.client.login(username='b', password='p')
 self.category = Category.objects.create(name='Health')

 def test_create_budget(self):
 resp = self.client.post('/finance/budgets/create/', {'category': self.category.id,
 'monthly_limit': '150.00'}, follow=True)
 self.assertRedirects(resp, '/finance/budgets/')
 self.assertTrue(Budget.objects.filter(user=self.user, category=self.category, monthly_limit='150.00').exists())
```



```
def test_unique_budget_per_user_category(self): Budget.objects.create(user=self.user, category=self.category,
monthly_limit='100')

resp = self.client.post('/finance/budgets/create/', {'category': self.category.id,

'monthly_limit': '200'})

self.assertContains(resp, 'Budget with this User and Category already exists', status_code=200)

'''
```

Note: the default model `unique_together` will raise `IntegrityError` at DB level; better to add form validation to catch and show user-friendly errors. Add form `clean` to check existing budget.

#### 10. form validation for uniqueness (optional)

```
'''python

class BudgetForm(forms.ModelForm):
 class Meta:
 model = Budget
 fields = ['category', 'monthly_limit']

 def __init__(self, *args, **kwargs):
 self.user = kwargs.pop('user', None)
 super().__init__(*args, **kwargs)

 def clean(self):
 cleaned = super().clean()
 category = cleaned.get('category')
 if self.user and Budget.objects.filter(user=self.user, category=category).exists():
 raise forms.ValidationError("You already have a budget for this category.")
 return cleaned

'''
```

And adjust view to pass user to form on POST: `form = BudgetForm(request.POST, user=request.user)`

#### 11. run & verify

### Commands:

```
'''bash

python manage.py makemigrations finance
python manage.py migrate
python manage.py runserver
python manage.py test finance.tests.BudgetTests

'''
```

#### Notes and cautions

- DB uniqueness is final guard; always validate on form level to present friendly messages and to avoid `IntegrityError` on save. If saving without form validation, wrap in `try/except` to catch `IntegrityError` and show message.
- Consider currency precision and rounding rules for `monthly_limit` (`DecimalField` with appropriate `max_digits/decimal_places`). Use `Decimal` in Python.

Example 3 complete.

```

```

## Final notes for all three examples

- Always run `python manage.py makemigrations` and inspect generated migration before applying in production. Use `python manage.py sqlmigrate appname migration_number` to preview SQL and assess lock/impact.
- Write tests for model behavior (validation), view behavior (permission/redirects), and integration (file storage).
- For file uploads in tests, use `SimpleUploadedFile` and make sure `MEDIA_ROOT` during tests points to temp dir; Django's `TestCase` isolates DB but not media storage — you may need to override `settings.MEDIA_ROOT` in tests to a temp dir and cleanup.
- Add new apps (if you used `startapp`) to `INSTALLED_APPS` then run migrations.
- When changing templates, ensure static references use `{% load static %}` and `{% static`

`'path' %}`.

What you should be able to do now

- Implement a `OneToOne` Profile with `ImageField`, wire up forms/views/templates, and test upload handling.
- Add GET-based search/filtering to list views with safe ORM usage (`select_related/prefetch_related`) and paginate results.
- Add a new Budget model, ensure uniqueness, create forms/views/templates, and write tests covering DB uniqueness and form validation.

If you want, I can now:

- (A) Generate the exact SQL produced by `sqlmigrate` for each migration file (full text) for your DB version (I can produce plausible PostgreSQL SQL as examples as shown).
- (B) Provide small improvements: signal-free profile auto-creation alternative (create profile on first access), disk cleanup of old avatars on change, thumbnail generation (`django-imagekit`), and S3 storage example (`django-storages` config).
- (C) Move to Prompt 3 (Templates/Static/Media + Auth + Middleware) and generate that full code-first content as requested.

Tell me which of A/B/C you want next (or "done" if this meets your needs). -----

---

Got it — I'll continue as your Senior Django Architect/Mentor and deliver Prompt 3 exactly as requested: code-first, practical, and interview-ready. After each Part I'll give 2 short predict-then-reveal questions (you should try to answer them before reading the revealed answers).

## PART 1 — TEMPLATES, STATIC, MEDIA

1. Minimal template file (DTL) — real code and literal explanation File: `templates/finance/tx_detail.html`

```django

```
{% extends "base.html" %}
```

```
{% load static %}
```

```
{% block content %}
```

Transaction {{ tx.id }}

```
Date: {{ tx.date }}
```

```
Amount: {{ tx.amount }}
```

```
Category: {{ tx.category.name }}
```

```
{% if tx.receipt %}
```

```
Receipt: receipt
```

```
{% else %}
```

```
No receipt
```

{% endif %} {% endblock %} ## `` What each piece does, literally - {% extends "base.html" %} — tells DTL to inherit base.html (template inheritance). - {% load static %} — loads the static template tag library so you can use {% static 'path' %}. - {% block content %} ... {% endblock %} — defines/overrides a named region from base template. - {{ tx.amount }} — variable interpolation: inserts the value of tx.amount escaping HTML by default. - {% if tx.receipt %} ... {% endif %} — a template tag controlling flow (condition). - {% static 'css/finance.css' %} — returns the URL path for a static asset.

DTL vs Jinja2 — key differences (explicit)

- Syntax is similar in many places ({{ % }} and {{ }}), but:
- DTL auto-escapes variables by default; Jinja2 can be different depending on config.
- DTL has fewer built-in Python-like expressions (no arbitrary function calls). Jinja is more Pythonic and allows more logic in templates.
- DTL template tag libraries are pluggable; Jinja templates use environment/global functions.
- Many Django-specific tags (e.g., {% url %}, {% static %}, {% csrf_token %}) and context processors are DTL-first (Jinja can be used but needs explicit config).
- Practical rule: prefer DTL for Django apps unless you need Jinja features; DTL encourages keeping logic out of templates which is good for maintainability and security.

2. Full render flow (code + literal handoffs) Sequence and minimal code at each step:

View (views.py)

```
``python
```

```
from django.shortcuts import render, get_object_or_404 from .models import Transaction
```

```
def tx_detail(request, pk): tx = get_object_or_404(Transaction.objects.select_related('category'), pk=pk, user=request.user) return render(request, 'finance/tx_detail.html', {'tx': tx})
```

```
'''
```

Render flow (step-by-step):

- Browser requests URL GET /finance/transactions/42/
- URL resolver matches pattern and calls tx_detail(request, pk=42)
- View builds context: {'tx': tx} where tx is a Transaction model instance (QuerySet executed here)
- render() calls template loader to find templates/finance/tx_detail.html and base.html
- Template engine compiles template (cached) and renders with the context and context processors (e.g., request, user)
- Inside template, `{% static 'css/finance.css' %}` is replaced with a URL like

/static/css/finance.css

- HTML response sent back to browser; browser requests linked static CSS/JS files (separate HTTP requests) and renders page

ASCII diagram:

```
```text
```

Browser -> Django URL resolver -> view(tx\_detail) -> ORM -> context -> template engine -> HttpResponse -  
 > Browser Browser -> (separate) GET /static/css/finance.css

```
'''
```

### 3. Static files vs Media files — settings and literal differences

settings.py (real config)

```
```python
```

```
from pathlib import Path
BASE_DIR = Path(__file__).resolve().parent.parent
```

Static files (dev: project-owned CSS/JS/images)

```
STATIC_URL = '/static/'
STATICFILES_DIRS = [BASE_DIR / "assets"] # development assets folder
STATIC_ROOT = BASE_DIR / "staticfiles" # target for collectstatic in production
```

Media files (user uploads)

```
MEDIA_URL = '/media/'
MEDIA_ROOT = BASE_DIR / "media"
```

```
'''
```

Literal meaning

- STATICFILES_DIRS: locations where `collectstatic` (and dev static finder) looks for your app/project static files.

- `STATIC_ROOT`: where `collectstatic` copies all static assets for production serving (e.g., by Nginx or WhiteNoise).
- `MEDIA_ROOT`: filesystem path where uploaded user files are saved.
- `MEDIA_URL`: URL prefix to serve uploaded files.

Where to put what

- Project CSS/JS/images you author — put under `assets/` or `app_name/static/app_name/`.
 - User uploads (receipts, avatars) — saved via `FileField/ImageField` into `MEDIA_ROOT`.
4. "Add a new image to a page" — exact minimal changes (static image) Goal: show a marketing image or icon in a template.

Files & actions:

- Place image file on disk:
- `assets/img/finance-hero.png` (developer-owned)
- Reference in template:

```
``django
```

```
{% load static %} hero
```

```
```
```

Dev behavior

- In `DEBUG`, Django `staticfiles` app serves the file; `{% static %}` returns

`'/static/img/finance-hero.png'`.

Production behavior

- Run `python manage.py collectstatic` which copies `assets/img/finance-hero.png` into `STATIC_ROOT/img/finance-hero..png` (if you use hashed storage), and Nginx/WhiteNoise serves it from `STATIC_ROOT`.

5. "User uploads an image (receipt photo)" — full code and physical file path

## Model (models.py)

```
``python
```

```
from django.db import models
```

```
class Transaction(models.Model):
 user = models.ForeignKey(settings.AUTH_USER_MODEL,
on_delete=models.CASCADE)
 amount = models.DecimalField(max_digits=12, decimal_places=2)
 date = models.DateField()
 receipt = models.ImageField(upload_to='receipts/%Y/%m/%d/', null=True, blank=True)
```

```
```
```

Form (forms.py)

```
```python
```

```
from django import forms from .models import Transaction
```

```
class ReceiptForm(forms.ModelForm): class Meta: model = Transaction fields = ['receipt']
```

```
```
```

View (views.py)

```
```python
```

```
from django.shortcuts import render, get_object_or_404, redirect from django.contrib.auth.decorators import login_required from .forms import ReceiptForm
```

### @login\_required

```
def upload_receipt(request, pk): tx = get_object_or_404(Transaction, pk=pk, user=request.user) if request.method == 'POST': form = ReceiptForm(request.POST, request.FILES, instance=tx) if form.is_valid(): form.save() return redirect('finance:transaction_detail', pk=pk) else: form = ReceiptForm(instance=tx) return render(request, 'finance/upload_receipt.html', {'form': form, 'tx': tx})
```

```
```
```

Template (templates/finance/upload_receipt.html)

```
```django
```

```
{% extends "base.html" %} {% block content %}
```

## Upload receipt for transaction {{ tx.id }}

---

```
{% csrf_token %} {{ form.as_p }} ##
```

```
{% endblock %} ## ``` Where the file ends up physically - Given MEDIA_ROOT = /path/to/project/media, and upload_to='receipts/%Y/%m/%d/', a file uploaded on 2026-06-23 with original name photo.jpg will be saved roughly at: ## /path/to/project/media/receipts/2026/06/23/photo.jpg - Database stores path like 'receipts/2026/06/23/photo.jpg' in Transaction.receipt.
```

### Serving in dev vs prod

- Dev: If DEBUG=True and you added urlpatterns += static(...) in project urls, Django serves media files.
- Prod: Nginx or CDN/S3 should serve files. Example Nginx location:

```
```nginx
```

```
location /media/ { alias /srv/myproject/media/;
```

```
}
```

```
```
```

Or use S3 via django-storages: settings for S3 media (example):

```
```python
```

```
DEFAULT_FILE_STORAGE = 'storages.backends.s3boto3.S3Boto3Storage' AWS_STORAGE_BUCKET_NAME =  
'my-bucket'
```

AWS credentials via env vars

```
MEDIA_URL = 'https://my-bucket.s3.amazonaws.com/'
```

```
```
```

6. collectstatic — what it does to files on disk (concrete)

### Command:

```
```bash
```

```
python manage.py collectstatic --noinput
```

```
```
```

What it does, literally

- Scans STATICFILES\_DIRS and each app's static// directories.
- Copies (or stores via configured storage backend) each discovered file into STATIC\_ROOT (or remote storage).
- If using ManifestStaticFilesStorage, generates hashed filenames and a manifest file staticfiles.json mapping original names to hashed names. This prevents stale caching. Example before collectstatic:

```
```
```

```
project/ assets/css/site.css apps/finance/static/finance/js/main.js
```

```
```
```

After collectstatic (STATIC\_ROOT = project/staticfiles):

```
```
```

```
project/staticfiles/css/site.css project/staticfiles/finance/js/main.3f2a1d.js # hashed if enabled  
project/staticfiles/staticfiles.json # manifest
```

```
```
```

Why hashed names?

- For long-lived CDN caching: URL changes when content changes so browsers pick up new version.

## WhiteNoise (simple production static serving)

- pip install whitenoise
- Add 'whitenoise.middleware.WhiteNoiseMiddleware' high in MIDDLEWARE (after SecurityMiddleware) and set STATICFILES\_STORAGE to 'whitenoise.storage.CompressedManifestStaticFilesStorage' (or similar).
- WhiteNoise serves the files from your app, no separate Nginx required for simple deployments.

## Summary table static vs media

### - Static

- Who owns it: developers
- Example: CSS/JS/theme images
- Settings: STATICFILES\_DIRS, STATIC\_ROOT, STATIC\_URL
- Served by: collectstatic -> STATIC\_ROOT + Nginx/WhiteNoise/CDN

### - Media

- Who owns it: users
- Example: uploaded receipts, avatars
- Settings: MEDIA\_ROOT, MEDIA\_URL, DEFAULT\_FILE\_STORAGE
- Served by: direct server/NGINX or cloud storage (S3) with proper permissions

Part 1 — QUIZ (predict-then-reveal) Q1: If you add a file assets/img/logo.png and reference it via `{% static 'img/logo.png' %}`, what command(s) must you run/deploy steps must you take so the image is served in production? Try to predict (files changed and commands). Answer below.

Q2: A user uploads a receipt named receipt.jpg. Given MEDIA\_ROOT=/srv/myproject/media and upload\_to='receipts/%Y/%m/%d/', what will be the stored DB value and the absolute filesystem path for a 2026-06-23 upload? Predict, then reveal below.

Reveal answers — check after you predict:

### A1:

- Locally: no command required for DEBUG=True dev server (dev static finders serve it), but in production you must run:
- python manage.py collectstatic (this will copy assets/img/logo.png into STATIC\_ROOT/img/logo.png or create a hashed filename if using manifest storage).
- Deploy static files (ensure STATIC\_ROOT is accessible by Nginx or WhiteNoise) and confirm Nginx/WhiteNoise configured to serve STATIC\_URL.
- Also ensure STATICFILES\_DIRS includes assets/ (so collectstatic finds the file).

### A2:

- DB value stored: 'receipts/2026/06/23/receipt.jpg'
- Absolute filesystem path: /srv/myproject/media/receipts/2026/06/23/receipt.jpg (assuming MEDIA\_ROOT = /srv/myproject/media)



---

## PART 2 — AUTHENTICATION

1. Full login flow with concrete code (login form → authenticate() → login() → session cookie → subsequent request) Login form using Django's built-in view (quick option)

- Built-in view: `django.contrib.auth.views.LoginView` — recommended unless you need custom behavior.

Project urls:

```
```python
```

project urls.py

```
from django.urls import path from django.contrib.auth import views as auth_views
```

```
urlpatterns = [
```

```
path('accounts/login/', auth_views.LoginView.as_view(template_name='registration/login.html'), name='login'),
path('accounts/logout/', auth_views.LogoutView.as_view(next_page='/'), name='logout'),
```

```
]
```

```
```
```

### Template (registration/login.html)

```
```django
```

```
{% csrf_token %} {{ form.username.label_tag }} {{ form.username }} {{ form.password.label_tag }} {{
form.password }}  ##
```

```
## ``` What happens step-by-step (internals) - User submits POST /accounts/login/ with username/password
and CSRF token. - LoginView validates form and calls django.contrib.auth.authenticate(request, username,
password). - authenticate() iterates authentication backends (e.g., ModelBackend), loads the User and verifies
the password hash; if ok returns a User object. - login(request, user) — sets session data:
request.session[SESSION_KEY] = user.pk; backend path stored in session; eventually calls request.session.save()
which stores session data according to SESSION_ENGINE (default: DB table django_session). - Server responds
with Set-Cookie: sessionid=; browser stores cookie. - Subsequent request: browser sends Cookie: sessionid=;
SessionMiddleware uses sessionid to load session data (from DB/cache) and sets request.session;
AuthenticationMiddleware looks up user from session data and sets request.user to the authenticated user.
```

Key settings & where session lives

- `SESSION_ENGINE` (default 'django.contrib.sessions.backends.db') — session storage backend
- `SESSION_COOKIE_NAME` (default 'sessionid') — cookie name
- Sessions table: `django_session` (stores `session_key`, `session_data`, `expire_date`) if DB backend used

2. Code for custom login view (explicit usage of authenticate and login)

```
```python
```

```
from django.contrib.auth import authenticate, login
from django.shortcuts import render, redirect
from django.contrib import messages
```

```
def my_login_view(request):
 if request.method == 'POST':
 username = request.POST.get('username')
 password = request.POST.get('password')
 user = authenticate(request, username=username, password=password)
```

```
 if user is not None:
 login(request, user) # sets session and request.user
 return redirect('finance:dashboard')
 else:
 messages.error(request, "Invalid credentials")
 return render(request, 'login.html')
```

```
```
```

3. Session-based auth vs JWT — code/config differences and when to use each

Session-based (Django default)

- Uses server-side session storage and cookie (sessionid) to identify user.
- Code: `authenticate()`, `login()`, `request.user` via `AuthenticationMiddleware`.

- Pros:

- Simple to implement (built-in).
- Server controls session lifetime and can revoke sessions (delete session on server).
- CSRF protection works naturally for browser forms.

- Cons:

- Not ideal for public APIs consumed by third-party clients (mobile apps) unless you implement token adapters.
- Requires session store scaling (DB/Cache/redis).

JWT (JSON Web Token) example (DRF + simplejwt)

- Setup (pip install `django-rest-framework-simplejwt`), urls:

```
```python
```

```
from rest_framework_simplejwt.views import TokenObtainPairView, TokenRefreshView
```

```
urlpatterns += [
 path('api/token/', TokenObtainPairView.as_view(), name='token_obtain_pair'),
 path('api/token/refresh/', TokenRefreshView.as_view(), name='token_refresh'),
```

```
]
```

```
```
```

Client obtains access and refresh tokens, then includes `Authorization: Bearer <access_token>` header in subsequent API requests. Backend validates token signature and expiry; typically no server-side session lookup.

Differences in practice

- Session: cookie-based; CSRF required for unsafe HTTP methods from browsers; better for server-rendered pages.
- JWT: stateless (unless you keep a denylist/refresh store); useful for mobile/third-party clients, microservices, and when you want token-based auth in APIs.

When to use which

- Use session-based auth for server-rendered sites (regular Django templates) where users authenticate via browser.
- Use JWT for API-first services consumed by mobile/native apps, or when you need stateless auth across multiple services. Consider security tradeoffs: token revocation, refresh handling, token theft risk.

4. Decorators: `@login_required` and `permission_required` (real usage) `login_required` (views.py)

```
```python
```

```
from django.contrib.auth.decorators import login_required
```

```
@login_required(login_url='/accounts/login/')
```

```
def dashboard(request):
```

## request.user is guaranteed to be an authenticated user

---

```
 return render(request, 'finance/dashboard.html')
```

```
```
```

`permission_required` (for permission checks)

```
```python
```

```
from django.contrib.auth.decorators import permission_required
```

```
@permission_required('finance.change_transaction', raise_exception=True) def edit_transaction(request, pk):
```

## user must have 'finance.change\_transaction' permission

---

```
...
```

```
```
```

Under the hood:

- `@login_required` checks `request.user.is_authenticated` and redirects to `login_url` if not.
- `@permission_required` uses the permission checking backend, which typically checks `user.has_perm('app.codename')`, and can raise `PermissionDenied` or redirect.

Part 2 — QUIZ (predict-then-reveal) Q1: After login(request, user) is called, where is the user identity stored on the server and what cookie does the browser receive? Predict and then reveal below.

Q2: For an API consumed by a mobile app, why might session-based auth be awkward and JWT be preferable? Predict then reveal.

Reveal answers — check after you predict:

A1:

- Server-side: session data stored according to SESSION_ENGINE — commonly in DB table django_session where session_key maps to a serialized session containing user id and backend path.
- Browser receives Set-Cookie: sessionid=<session_key>; subsequent requests include Cookie: sessionid=<session_key>.

A2:

- Session-based auth requires browser cookies and CSRF protection, and server-side session storage; mobile apps must manage cookies and cannot easily rely on browser CSRF flow. JWT gives a bearer token that the mobile client can store and include in Authorization headers; it's stateless (no server session lookup) and naturally fits token-based API auth. But it introduces token revocation and security complexity.

PART 3 — MIDDLEWARE

1. MIDDLEWARE list example (settings.py) and execution order Example MIDDLEWARE (practical order)

```
python
```

```
MIDDLEWARE = [
```

```
'django.middleware.security.SecurityMiddleware', 'whitenoise.middleware.WhiteNoiseMiddleware', # if using
WhiteNoise 'django.contrib.sessions.middleware.SessionMiddleware',
'django.middleware.common.CommonMiddleware', 'django.middleware.csrf.CsrfViewMiddleware',
'django.contrib.auth.middleware.AuthenticationMiddleware',
'django.contrib.messages.middleware.MessageMiddleware',
'django.middleware.clickjacking.XFrameOptionsMiddleware',
```

custom middleware last typically, or wherever required

```
'finance.middleware.RequestTimerMiddleware',
```

```
]
```

```
python
```

Execution order & diagram

- Request phase: run top → bottom
- Response phase: run bottom → top

ASCII diagram:

```
```text
```

## Incoming Request

-> SecurityMiddleware (request) -> WhiteNoiseMiddleware (request) -> SessionMiddleware (request) -> CommonMiddleware (request) -> CsrfViewMiddleware (request) -> AuthenticationMiddleware (request)

-> ... (view)

<- AuthenticationMiddleware (response) <- CsrfViewMiddleware (response) <- CommonMiddleware (response) <- SessionMiddleware (response) <- WhiteNoiseMiddleware (response) <- SecurityMiddleware (response)

## Outgoing Response

```
```
```

Why ordering matters (concrete)

- SessionMiddleware must run before AuthenticationMiddleware because AuthenticationMiddleware needs request.session to find user id and load user.
- CsrfViewMiddleware must run before view to check CSRF tokens for unsafe requests.
- SecurityMiddleware (e.g., for SECURE_SSL_REDIRECT) should run early to enforce security headers/redirects.

2. Pick 3 built-in middleware — show what they do literally

a) SessionMiddleware

- What it does: loads session data into request.session at request time and saves session back at response time.
- Internals: looks up session key from cookie (SESSION_COOKIE_NAME), loads session store (DB/cache), attaches session dict-like object to request; on response, if session modified or new, sets session cookie and persists data.
- Practical effect: request.session['cart'] = [1,2] persists across requests for that session key.

b) AuthenticationMiddleware

- What it does: uses session info to populate request.user with a User object (or AnonymousUser).
- Internals: reads session data for user id and uses configured auth backend to get user, sets request._cached_user and request.user property.
- Practical effect: `request.user.is_authenticated` available and used by @login_required and other logic.

c) CsrfViewMiddleware

- What it does: for unsafe HTTP methods (POST/PUT/PATCH/DELETE), validates CSRF token in request against session or cookie-based token.
- Internals: looks for token in request.POST['csrfmiddlewaretoken'] or X-CSRFToken header for AJAX; raises 403 if not valid.
- Practical effect: protects forms from cross-site request forgery; templates must include `{% csrf_token %}` inside POST forms.

3. Minimal custom middleware (real code) — timing header example File: `finance/middleware.py`

```
python
```

```
import time from django.utils.deprecation import MiddlewareMixin # for old-style; modern preferred:
```

init/call

Modern style (recommended)

```
class RequestTimerMiddleware:
    def init(self, get_response):
        self.get_response = get_response

    def call(self, request):
        start = time.perf_counter()
        response = self.get_response(request) # call next middleware
        / view duration = (time.perf_counter() - start) * 1000.0
```

add a header for debugging (avoid in prod)

```
response['X-Request-Duration-ms'] = f'{duration:.2f}'
return response
```

Async-aware middleware example

```
class AsyncExampleMiddleware:
    def init(self, get_response):
        self.get_response = get_response

    async def call(self, request):
```

you can await things here if needed

```
response = await self.get_response(request)
response['X-Async'] = 'true'
return response
```

```
...
```

Explain each part literally

- **init**(self, get_response): called once at server startup; `get_response` is callable to run next middleware/view.
- **call**(self, request): executed per-request; here you run code before and after calling `get_response`.
- In async middleware, **call** can be async to await asynchronous operations.

Where to add it

- Add 'finance.middleware.RequestTimerMiddleware' to MIDDLEWARE; location dictates when it runs in the chain.

Common beginner mistakes with middleware

- Doing heavy/blocking I/O in middleware (e.g., slow DB calls) which increases latency for all requests.
- Misordering middleware (e.g., putting AuthenticationMiddleware before SessionMiddleware).
- Using old-style process_request/process_response hooks incorrectly; prefer new callable-style.

Part 3 — QUIZ (predict-then-reveal) Q1: If you put AuthenticationMiddleware before SessionMiddleware in the MIDDLEWARE list, what breaks? Predict then reveal.

Q2: You want to add a middleware that forbids requests lacking a custom header X-Client-Id. Where should that middleware go (before/after AuthenticationMiddleware) and why? Predict then reveal.

Reveal answers — check after you predict:

A1:

- AuthenticationMiddleware will fail to find request.session (it expects SessionMiddleware to have already attached request.session), so request.user will not be properly set (or it may behave unpredictably). This breaks login/session-based auth and decorators relying on request.user.

A2:

- Put the header-checking middleware before AuthenticationMiddleware if you want to reject unauthenticated requests early (and avoid unnecessary session/user lookups). If header is required for authentication itself (e.g., header contains token used to authenticate and you need that to set request.user), put it before or as part of an authentication middleware. If the header is just an additional check after auth, put it after AuthenticationMiddleware. In

summary: place it where its inputs are available—before AuthenticationMiddleware to short-circuit and reject early, or after if it depends on request.user.

— End of Prompt 3 output.

If you want, next steps I can take:

- Provide a short checklist for migrating media to S3 (django-storages) with exact settings and an example CI/CD deploy snippet.
- Add example Nginx config snippets for serving STATIC_ROOT and MEDIA_ROOT.
- Generate 3 short, targeted interview answers (30s / 2m / deep) for Templates/Static/Media, Auth, Middleware to practice speaking about your project.

Which of those (if any) do you want next? -----

Great — I'll continue as your Senior Django Architect/Mentor and give you a compact, code-first DRF bridge, interview-ready answers at three depths, and a dense revision cheatsheet you can memorize and use in interviews. I anchor everything to your finance-tracker (transactions, categories, budgets, receipts).

PART 1 — DRF BRIDGE

1. Side-by-side: server-rendered view vs DRF endpoint (transactions list)

Django template version (server-rendered)

```
```python
```

## finance/views.py (template)

---

```
from django.shortcuts import render from django.contrib.auth.decorators import login_required from .models import Transaction
```

```
@login_required
```

```
def transaction_list(request): qs = Transaction.objects.filter(user=request.user).select_related('category').order_by('-date')[:50] return render(request, 'finance/transaction_list.html', {'transactions': qs})
```

```
```
```

What it does: builds a QuerySet, renders HTML with template using tx fields.

DRF equivalent — Serializer + ViewSet (API)

```
```python
```

## finance/api/serializers.py

---

```
from rest_framework import serializers from finance.models import Transaction
```

```
class TransactionSerializer(serializers.ModelSerializer): category = serializers.CharField(source='category.name', read_only=True) class Meta: model = Transaction fields = ('id', 'date', 'amount', 'note', 'category', 'receipt_url')
```

```
receipt_url = serializers.SerializerMethodField() def get_receipt_url(self, obj): request = self.context.get('request') return request.build_absolute_uri(obj.receipt_url) if obj.receipt else None
```

```
```
```

```
```python
```

## finance/api/views.py

---



```

from rest_framework import viewsets, permissions
from finance.models import Transaction
from .serializers import TransactionSerializer

class TransactionViewSet(viewsets.ReadOnlyModelViewSet):
 serializer_class = TransactionSerializer
 permission_classes = [permissions.IsAuthenticated]

 def get_queryset(self):
 return Transaction.objects.filter(user=self.request.user).select_related('category').order_by('-date')

```

```
```python
```

finance/api/urls.py

```

from rest_framework.routers import DefaultRouter
from .views import TransactionViewSet

router = DefaultRouter()
router.register('transactions', TransactionViewSet, basename='transaction')
urlpatterns = router.urls

```

Line-by-line mapping:

- Template view builds QuerySet → DRF ViewSet `get_queryset()` returns same QuerySet.
- Template context variables (`tx.amount`, `tx.category.name`) → Serializer fields (`amount`, `category` via `source`).
- Rendering template to HTML → DRF serializes to JSON (Response handled by DRF renderers).
- `@login_required` → `permission_classes = [IsAuthenticated]`
- URL patterns: project urls include app url vs DRF router registers viewset endpoints

(`/api/transactions/`).

2. DRF building blocks — minimal code + what/why/how

Serializer (ModelSerializer)

```
```python
```

```

class TransactionSerializer(serializers.ModelSerializer):
 class Meta:
 model = Transaction
 fields = ['id', 'date', 'amount', 'note', 'category']

```

- What: converts model instances to primitive types and validates input for create/update.
- Why: separates data shape/validation from views.
- Internals: builds field mappings from `model _meta`; `.save()` calls create/update methods.

APIView vs ViewSet

- APIView (low-level): gives full control; you implement methods for HTTP verbs.

```
```python
```

```
from rest_framework.views import APIView
from rest_framework.response import Response

class TransactionListAPI(APIView):
    permission_classes = [IsAuthenticated]

    def get(self, request):
        qs = Transaction.objects.filter(user=request.user)
        serializer = TransactionSerializer(qs, many=True, context={'request': request})
        return Response(serializer.data)

```
```

- ViewSet (higher-level): maps actions (list/create/retrieve/update/destroy) to methods and integrates with routers. Use when you want RESTful resource endpoints quickly.

## Router (urls)

```
```python
```

```
from rest_framework.routers import DefaultRouter
router = DefaultRouter()
router.register('transactions', TransactionViewSet)
urlpatterns = router.urls

```
```

- What: creates standard REST endpoints automatically (list, retrieve, create, update, destroy) with conventional URL patterns and names.

## Permissions (example)

```
```python
```

```
from rest_framework.permissions import IsAuthenticated

permission_classes = [IsAuthenticated]

```
```

- What: restricts access; run check before view logic. Implement custom permission by subclassing BasePermission and implementing has\_permission / has\_object\_permission.

Throttling (simple example)

```
```python
```

settings.py

```
REST_FRAMEWORK = {
    'DEFAULT_THROTTLE_CLASSES': [
```

```
'rest_framework.throttling.UserRateThrottle',

],

'DEFAULT_THROTTLE_RATES': {

'user': '1000/day',

}

}

'''
```

- What: rate-limits requests per user/IP.
- Use-case: public APIs to guard abuse.

Authentication (DRF)

- DRF reuses Django authentication; default is SessionAuthentication and BasicAuthentication. For token/JWT you configure TokenAuthentication or custom.

3. What stays identical vs what's new

Stays identical

- Models: Transaction, Category, Budget remain the same.
- DB/ORM: QuerySets, select_related/prefetch, migrations unchanged.
- Core auth concepts: authenticate, login, request.user available if SessionAuthentication used.

New / different

- Serializers: explicit data contracts, validation separate from forms.
- Request parsing and content negotiation: DRF parses JSON, multipart, form data; renders

JSON/XML.

- Response objects: DRF Response with renderer selection; not render() templating.
- Routers/ViewSets: automatic RESTful route generation.
- Browsable API: DRF provides interactive HTML for API exploration.
- Permissions/Throttling/Authentication: DRF-specific classes to apply across API.

PART 2 — INTERVIEW MODE

For each topic: 30-second, 2-minute, and advanced answer with finance-tracker examples.

A) MVT architecture

- 30s:

"Django uses MVT: Models (DB schema), Views (request/business logic), Templates (presentation). URL routes call Views, Views use Models and feed Templates to render

HTML."

Example: Transaction model → transaction_list view → transaction_list.html renders a table.

- 2m:

"Explain flow: URL -> view -> view queries Transaction.objects.filter(user=request.user) (Model/ORM) -> builds context -> render(template, context). The view should be thin; business logic belongs in services or model methods. Templates use context variables and template tags for presentation. For APIs with DRF, the view returns serialized JSON rather than HTML. Use select_related/prefetch to avoid N+1 when templates iterate over relationships."

- Advanced:

"Discuss tradeoffs: MVT promotes separation; but when doing heavy business logic (budget forecasting, anomaly detection), extract into service layer or async tasks. For testability, keep views as orchestration only. Show prove: run python manage.py shell, build QuerySet for transactions and call .explain() to inspect SQL plan; show sample code where moving logic into service reduces code duplication across views and API endpoints."

B) Full request lifecycle (Browser to Response)

- 30s:

"Browser → DNS/TCP/TLS → web server → WSGI/ASGI → Django middleware → URL resolver → view → ORM/services → template/serializer → HttpResponse → middleware response → client."

- 2m:

"Walk through example: request for /finance/transactions triggers AuthenticationMiddleware (session loaded), URLConf finds transaction_list, view executes and performs QuerySet (SQL sent to DB), then render; response middleware adds security headers before returning. For performance debug, capture which layer is slow (db, template, external API) and use logging, Django debug toolbar, or EXPLAIN ANALYZE to inspect DB."

- Advanced:

"Proof-level discussion: show how to measure time in middleware (RequestTimerMiddleware), capture SQL queries via connection.queries or Django DB instrumentation. Show an example trace: middleware logs zero wait, view spends 120ms, ORM query 110ms (SELECT with sequential scan) — remedy: add index or rewrite query. For async/WSGI differences: ASGI allows long-lived connections and websockets

(channels)."

C) ORM / N+1 problem

- 30s:

"N+1: when you query a list (1 query) and then access a relation per item (N additional queries). Fix with select_related for FK/OneToOne or prefetch_related for M2M/reverse."

- 2m:

"Example: rendering transactions and showing category name. Naive loop causes N+1: 1 query for transactions + N queries for categories. Fix: `Transaction.objects.filter(...).select_related('category')` makes one JOIN query. For tags (M2M), use `prefetch_related('tags')`, which issues 1 extra query to load all tags for listed transactions."

- Advanced:

"Show SQL: naive -> `SELECT ... FROM finance_transaction WHERE user_id=...`; then for each tx `SELECT ... FROM finance_category WHERE id=...`. Using `select_related` yields a LEFT JOIN; `prefetch` issues a second query with `WHERE transaction_id IN (...)` then assigns via Python. Evaluate memory vs SQL cost: `prefetch` pulls more data client-side; for huge lists prefer pagination and partial fields via `only()` or `values()`. Use `QuerySet.explain()` to inspect query plan and index usage."

D) Migrations

- 30s:

"`makemigrations` inspects model state and generates migration files; `migrate` applies them to DB."

- 2m:

"Describe migration file: operations list (`CreateModel`, `AddField`, `RunPython`). For non-blocking deploys: adopt `expand/backfill/contract` pattern (add nullable field, backfill with `RunPython`, then set NOT NULL). Use `sqlmigrate` to preview generated SQL and watch for table rewrites and locking."

- Advanced:

"Explain migration graph, dependencies and `django_migrations` table. Show how to squash migrations and the pitfalls (must preserve history and CI baseline). For zero-downtime deployments on large tables, use `pg_repack`/materialized views or background jobs, and careful index creation (`CREATE INDEX CONCURRENTLY` in Postgres via `RunSQL` with transactional caveats). Prove: show `sqlmigrate` output for `AlterField` that uses `COPY` for table rewrite."

E) Session vs JWT auth

- 30s:

"Session auth: server-side sessions + cookie — great for browser apps. JWT: stateless signed tokens — common for mobile/API clients."

- 2m:

"Session-based works well for your finance-tracker server-rendered UI: `login()` creates server-side session stored in DB/cache; CSRF middleware protects POSTs. For API consumed by mobile apps, JWT simplifies stateless auth: client stores token and sends Authorization header. JWT requires refresh/rotation, revocation strategy, and careful token lifetime management."

- Advanced:

"Discuss security tradeoffs: session revocation easy (delete server session), JWT revocation needs denylist or short-lived access + refresh token pattern. Demonstrate code for both: Django login() for session; DRF simplejwt TokenObtainPairView for JWT. Discuss session store scaling (use Redis for fast session lookups). For high-security scenarios (banking), use short-lived tokens, secure refresh endpoints, and detect refresh token misuse."

F) Middleware

- 30s:

"Middleware are hooks executed for every request/response to handle cross-cutting concerns (auth, sessions, CSRF, logging). Request-phase executes top-down, response-phase bottom-up."

- 2m:

"Explain order importance: SessionMiddleware must precede AuthenticationMiddleware. Use middleware for global behavior (e.g., security headers). For per-view behaviors, prefer decorators or mixins to avoid global performance impact."

- Advanced:

"Discuss async middleware differences and limitations. Show how to implement per-request tracing with middleware and integrate with observability (OpenTelemetry). Warn against blocking external calls in middleware and show pattern to enqueue background work post-response (transaction.on_commit hook + Celery)."

G) Django vs DRF: when to use which

- 30s:

"Use Django templating for server-rendered pages and admin; use DRF for API-first services or when mobile/third-party clients need JSON endpoints."

- 2m:

"Your finance tracker: keep server-rendered views for user dashboard if you prefer simple server-side rendering (sessions and CSRF are straightforward). For integrations (mobile app, external sync, public API), add DRF endpoints. DRF provides serializers, browsable API, versioning, throttling, and consistent error formats."

- Advanced:

"Discuss architecture: use DRF ViewSets for consistent REST resources, share service layer between template views and API endpoints to avoid duplicated business logic. For heavy analytics, expose read-only endpoints optimized separately (denormalized tables, caching, materialized views). Show sample migration path: create Serializer + APIView that wraps existing service layer, add router, protect with IsAuthenticated, and gradually version"

API."

PART 3 — DENSE REVISION CHEATSHEET

(Skimmable one-page style — keep this open as your memory aid)

A) Essential commands

- `python -m venv .venv`
- `source .venv/bin/activate`
- `pip install -r requirements.txt`
- `django-admin startproject projectname .`
- `python manage.py startapp appname`
- `python manage.py runserver`
- `python manage.py makemigrations`
- `python manage.py migrate`
- `python manage.py sqlmigrate app_name 000X`
- `python manage.py showmigrations`
- `python manage.py createsuperuser`
- `python manage.py shell`
- `python manage.py collectstatic --noinput`
- `python manage.py test`
- `python manage.py dumpdata app.Model > data.json`
- `python manage.py loaddata data.json`

B) Request flow diagram (compact)

```
```text
```

Client -> DNS -> TCP/TLS -> Nginx (reverse proxy) -> Gunicorn/Uvicorn -> WSGI/ASGI app -> Django startup (settings, app registry) already happened -> Middleware (request: top->down) -> URL resolver -> view (FBV/CBV/APIView/ViewSet) -> Business logic / service layer -> ORM queries -> DB -> Serializer or Template render -> HttpResponse -> Middleware (response: bottom->up)

-> **Server** -> **Client**

```
```
```

C) One-line purpose of each common file

- `manage.py`: per-project CLI entrypoint (sets `DJANGO_SETTINGS_MODULE`).
- `settings.py`: global configuration (apps, DB, middleware, static/media).
- `urls.py` (project): root URL router; include app urls.

- wsgi.py / asgi.py: production entrypoints for WSGI/ASGI servers.
- apps.py: AppConfig + ready() hook.
- models.py: ORM model classes -> DB schema.
- views.py: request handlers (thin orchestration).
- templates/: DTL files for HTML.
- static/: developer-owned assets before collectstatic.
- forms.py: form validation / ModelForm.
- admin.py: admin site registration/customization.
- middleware.py: global request/response hooks.
- migrations/: migration files (schema/data operations).

D) ORM quick-reference (method → what it does → SQL effect)

- `Model.objects.filter(**conds)` → `QuerySet` (`SELECT ... WHERE ...`) — no execute until evaluated.
- `.get(**conds)` → fetch single row (`LIMIT 1`) → raises `DoesNotExist/MultipleObjectsReturned`.
- `.exclude(...)` → `WHERE NOT ...`
- `.order_by('field')` → `ORDER BY ...`
- `.annotate(total=Sum('amount'))` → `SELECT ... GROUP BY ... , SUM(amount)`
- `.aggregate(Sum('amount'))` → `SELECT SUM(amount) ...`
- `.select_related('fk')` → add `JOIN` to `SELECT` (no extra queries)
- `.prefetch_related('m2m')` → extra query to fetch related rows then map in Python
- `.values('field')` → `SELECT` specific columns, returns dicts
- `.values_list('id', flat=True)` → `SELECT` column, return list of values
- `.only('a','b') / .defer('bigfield')` → load subset of columns
- `.update(...)` → single `SQL UPDATE ... WHERE ...` (no save hooks)
- `.delete()` → `SQL DELETE` (cascades per DB constraints)
- `.bulk_create([...])` → bulk `INSERT`, bypasses `save()`

E) Auth flow diagram (session-based)

```
```text
```

User posts credentials -> `authenticate()` verifies credentials -> `login(request,user)` -> server stores session (django\_session row) -> server returns `Set-Cookie: sessionid=` -> subsequent request includes `Cookie` -> `SessionMiddleware` loads session -> `AuthenticationMiddleware` sets `request.user`

```
```
```

F) Static vs Media (quick table)

- Static:

- Owner: developer
- Settings: STATIC_URL, STATICFILES_DIRS, STATIC_ROOT
- Deployment: collectstatic -> STATIC_ROOT -> served by Nginx/WhiteNoise/CDN

- Media:

- Owner: users (uploads)
- Settings: MEDIA_URL, MEDIA_ROOT, DEFAULT_FILE_STORAGE
- Deployment: served by Nginx or cloud storage (S3), require access controls as needed

G) DRF building blocks (one-line table)

- Serializer / ModelSerializer: defines JSON shape and validation (like Form for APIs).
- APIView: low-level class mapping HTTP verbs to methods; manual control.
- GenericAPIView + mixins: get implementations for list/create/retrieve/update/destroy.
- ViewSet / ModelViewSet: groups related actions and works with routers to generate URLs.
- Routers: auto-create RESTful route patterns (list/retrieve/etc).
- Authentication classes: SessionAuthentication, TokenAuthentication, JWT, etc.
- Permission classes: IsAuthenticated, IsAdminUser, custom BasePermission
- Throttling: UserRateThrottle, AnonRateThrottle; config in settings.
- Pagination: PageNumberPagination / LimitOffsetPagination; set in REST_FRAMEWORK settings.
- Parsers: JSONParser, FormParser, MultiPartParser (handle file uploads).
- Renderers: JSONRenderer,BrowsableAPIRenderer (HTML), XMLRenderer.

H) Quick DRF code snippets

- Register ViewSet with router (urls.py):

```
```python
```

```
from rest_framework.routers import DefaultRouter
router = DefaultRouter()
router.register('transactions', TransactionViewSet)
urlpatterns = [path('api/', include(router.urls)),
```

```
]
```
```

- Protect endpoint:

```
```python
```

```
class TransactionViewSet(viewsets.ModelViewSet):
 permission_classes = [IsAuthenticated]
```

```
```
```

I) What to say in interviews (1-line cheat)

- If asked "where to add a new endpoint?": add Serializer + ViewSet (or APIView) + register in router -> reuse service layer -> secure with permissions and throttling -> add tests.

- If asked "how to fix slow page": identify slow layer (SQL vs template vs external API). For SQL use `select_related/prefetch`, add indexes, use `EXPLAIN ANALYZE`; for template heavy-lifting, cache fragments; for external calls, background tasks.
- If asked "how to migrate schema safely": `expand/backfill/contract`; use `RunPython` to backfill; prefer concurrent index creation on Postgres.

WHAT YOU SHOULD BE ABLE TO DO AFTER THIS

- Convert a transaction-list view to DRF by writing `TransactionSerializer` and `TransactionViewSet`, register router, and secure with `IsAuthenticated`. Share the same `QuerySet` optimizations (`select_related/prefetch`).
- Explain MVT and the full request lifecycle in 30s/2min/deep forms, with concrete examples from your finance tracker.
- Detect and fix N+1, write safe migrations, choose appropriate auth (session vs JWT), and add middleware thoughtfully.
- Use the cheatsheet commands and patterns to implement and deploy new endpoints quickly.

If you want, next I can:

- Generate a ready-to-paste `TransactionSerializer` + `ViewSet` + router file tuned to your exact model fields (paste your `Transaction` model if you want exact names).
- Produce a 2-page printable PDF-style cheat sheet based on the dense revision section.
- Create 10 interview-style practice prompts (with model answers) you can rehearse aloud.

Which of those do you want next?